

Table of Contents

CLOUDCART: AWS-BASED MINI E-COMMERCE PLATFORM	5
CERTIFICATE	5
DECLARATION	6
ACKNOWLEDGEMENT	7
ABSTRACT	7
LIST OF FIGURES.....	8
LIST OF TABLES	8
LIST OF ABBREVIATIONS.....	9
1. INTRODUCTION.....	10
1.1 Introduction.....	10
1.2 Background	10
1.3 Motivation.....	11
1.4 Scope	12
1.5 Need of the Project	12
1.6 Existing Problems	13
1.7 Challenges.....	13
1.8 Need for Cloud-Based E-Commerce	14
1.9 Traditional E-Commerce Deployment	15
1.10 Limitations of the Existing System.....	15
1.11 Security Concerns in the Existing System	15
1.12 Cost Issues in the Existing System	16
1.13 Scalability Issues in the Existing System.....	16
1.14 Overview.....	16
1.15 Architecture of the Proposed System	17
1.16 Cloud-Based Deployment Model.....	17
1.17 Advantages of the Proposed System	17
1.18 Benefits Summary.....	18
1.19 Objectives	18
1.20 Introduction to the Literature Survey	19
1.21 Cloud Computing Fundamentals.....	19
1.22 Amazon Web Services Architecture and Best Practices	20
1.23 Web Application Development Technologies.....	20

1.24 E-Commerce System Design	21
1.25 Summary of Literature Survey.....	21
1.26 Technical Feasibility	22
1.27 Economic Feasibility	22
1.28 Operational Feasibility.....	22
1.29 Legal Feasibility	22
1.30 Schedule Feasibility.....	23
2. METHODOLOGY.....	23
2.1 Functional Requirements.....	23
2.2 Non-Functional Requirements	24
2.3 Software Requirements	24
2.4 Hardware Requirements	25
2.5 Overview	25
2.6 Activity Diagram – User Workflow	25
2.7 Sequence Diagram – Checkout Process	27
2.8 Workflow Narrative	27
2.9 Authentication Workflow.....	28
2.10 Cart Management Algorithm.....	28
2.11 Search Algorithm.....	29
2.12 Wishlist Algorithm.....	29
2.13 Checkout Process Algorithm.....	29
2.14 Session Handling Algorithm.....	30
3. TECHNOLOGY STACK	30
3.1 HTML5	30
3.2 CSS3	31
3.3 JavaScript (ES6)	31
3.4 PHP.....	32
3.5 MySQL.....	32
3.6 Apache HTTP Server.....	32
3.7 Amazon EC2.....	33
3.8 Amazon RDS	33
3.9 Amazon S3	34
3.10 AWS IAM	34
3.11 Amazon VPC.....	34

3.12 Security Groups	35
4. SYSTEM DESIGN / ARCHITECTURE.....	35
4.1 Overview	35
4.2 AWS Architecture Diagram	36
4.3 Component Diagram.....	36
4.4 Deployment Diagram	37
4.5 Data Flow Diagram – Level 0.....	38
4.6 Data Flow Diagram – Level 1.....	38
4.7 Class Diagram.....	39
4.8 Request Flow.....	40
4.9 Overview	41
4.10 Entity Relationship Diagram.....	41
4.11 Table Descriptions	42
4.12 Relationships.....	45
4.13 Normalization	45
4.14 Sample Records.....	46
4.15 Homepage Module	46
4.16 Authentication Module.....	46
4.17 Product Management Module.....	47
4.18 Search Module.....	47
4.19 Category Filter Module	47
4.20 Wishlist Module	47
4.21 Cart Module.....	47
4.22 Checkout Module.....	47
4.23 Orders Module	48
4.24 AWS Storage Module	48
4.25 User Profile Module.....	48
4.26 Session Management Module	48
4.27 Password Hashing.....	48
4.28 Prepared Statements	49
4.29 SQL Injection Prevention.....	49
4.30 Session Security	49
4.31 Authentication	49
4.32 Authorization	49

4.33 AWS IAM Security	49
4.34 VPC Security.....	49
4.35 Security Group Configuration.....	50
4.36 Principle of Least Privilege	50
5. IMPLEMENTATION.....	50
5.1 Launching the EC2 Instance.....	50
5.2 Configuring Apache.....	50
5.3 Installing PHP.....	51
5.4 Configuring MySQL Client Tools	51
5.5 Creating the Amazon RDS Instance	51
5.6 Connecting EC2 to RDS.....	51
5.7 Configuring IAM.....	51
5.8 Configuring VPC.....	51
5.9 Configuring Security Groups.....	52
5.10 Uploading Product Images to S3.....	52
5.11 Configuring File Permissions.....	52
5.12 Deploying the Application.....	52
5.13 Testing the Deployment.....	52
5.14 Testing Strategy	53
5.15 Test Cases.....	53
5.16 Testing Summary.....	56
6. RESULTS.....	56
6.1 Screenshots.....	56
6.2 Advantages	57
6.3 Limitations	58
7. CONCLUSION.....	59
7.1 Future Scope.....	59
7.2 Conclusion	60
APPENDIX A: REFERENCES.....	60
APPENDIX B: VIVA QUESTIONS.....	62

CLOUDCART: AWS-BASED MINI E-COMMERCE PLATFORM

A Project Report Submitted in Partial Fulfilment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

Name	Roll Number
Geethika	24P31A1254
Purnima Chopperla	24P31A1206
Thanuja	24P31A1222
Vardhan	24P31A1224
Pavan	24P31A1212
Nikhil	24P31A1243
Harshith	25P35A1205
Sriram	24P31A1201

Under the Guidance of

Mr. Juttuka Lovababu

DEPARTMENT OF INFORMATION TECHNOLOGY

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

2025 – 2026

CERTIFICATE

This is to certify that the project report entitled “**CloudCart: AWS-Based Mini E-Commerce Platform**” is a bonafide record of work carried out by **Geethika (24P31A1254), Purnima Chopperla (24P31A1206), Thanuja (24P31A1222), Vardhan (24P31A1224), Pavan (24P31A1212), Nikhil (24P31A1243), Harshith (25P35A1205) and Sriram (24P31A1201)** in partial fulfilment of the requirements for

the award of the degree of **Bachelor of Technology in Information Technology** from **Aditya College of Engineering & Technology**, during the academic year 2025–2026.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Project Guide

Mr. Juttuka Lovababu

Department of Information Technology

Head of the Department

Department of Information Technology

Aditya College of Engineering & Technology

External Examiner

DECLARATION

We hereby declare that the project work entitled “**CloudCart: AWS-Based Mini E-Commerce Platform**” submitted to the Department of Information Technology, Aditya College of Engineering & Technology, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, is an original work carried out by us under the guidance of **Mr. Juttuka Lovababu**, Project Guide.

We further declare that this project report has not been submitted, in full or in part, to any other University or Institute for the award of any degree, diploma, or similar title, and that all sources of information used in this report have been duly acknowledged.

Team Members

Name	Roll Number	Signature
Geethika	24P31A1254	
Purnima Chopperla	24P31A1206	
Thanuja	24P31A1222	
Vardhan	24P31A1224	
Pavan	24P31A1212	
Nikhil	24P31A1243	
Harshith	25P35A1205	
Sriram	24P31A1201	

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who has supported us during the course of this project work.

We are deeply thankful to our project guide, **Mr. Juttuka Lovababu**, Department of Information Technology, for his invaluable guidance, continuous encouragement, and constructive suggestions throughout the development of this project. His expertise and patient mentorship were instrumental in shaping the direction and quality of this work.

We extend our heartfelt thanks to the **Head of the Department, Department of Information Technology**, for providing the necessary infrastructure, resources, and a conducive environment for carrying out this project successfully.

We are also grateful to the **Principal and Management of Aditya College of Engineering & Technology** for their constant support and for providing state-of-the-art laboratory facilities that enabled us to explore cloud computing technologies in a practical setting.

We would like to thank all the faculty members of the Department of Information Technology for their valuable inputs during review sessions, which helped us refine our approach at every stage of this project.

Finally, we express our gratitude to our families and friends for their unwavering encouragement, patience, and moral support throughout the completion of this project.

Team Members Geethika, Purnima Chopperla, Thanuja, Vardhan, Pavan, Nikhil, Harshith, Sriram

ABSTRACT

Electronic commerce has become an integral part of modern retail, and the ability to deploy scalable, secure, and cost-effective online shopping platforms is a critical skill for aspiring software and cloud engineers. **CloudCart** is a mini e-commerce web application designed and deployed entirely on **Amazon Web Services (AWS)**, demonstrating the practical integration of web development technologies with cloud infrastructure services.

The system is built using **HTML5, CSS3, and JavaScript (ES6)** on the frontend, and **PHP** with a **MySQL** relational database on the backend. The application supports core e-commerce functionalities including user registration, login and logout, product browsing, keyword-based product search, category-based filtering, wishlist management, shopping cart operations, checkout, and order management.

The platform is hosted on an **Amazon EC2** instance running Apache and PHP, while persistent relational data is stored in **Amazon RDS (MySQL)**. Product images are stored and served through **Amazon S3**, providing durable and scalable object storage. Access to these resources is governed through **AWS Identity and Access Management (IAM)** roles

and policies following the principle of least privilege, while network isolation and traffic control are achieved through a custom **Amazon VPC** and finely tuned **Security Groups**.

This report documents the complete lifecycle of the CloudCart project — from problem identification and requirement analysis to system design, implementation, AWS deployment, and testing. It elaborates on the system architecture, database design, module-wise functionality, security mechanisms, and the deployment procedure followed to provision and configure the AWS environment. The project demonstrates how small and medium-scale e-commerce applications can be deployed on cloud infrastructure to achieve high availability, elastic scalability, reduced operational cost, and improved security compared to traditional on-premises hosting.

The successful implementation of CloudCart validates the feasibility of using AWS managed services to build a production-representative e-commerce solution within an academic setting, while also serving as a foundation for future enhancements such as payment gateway integration, AI-driven product recommendations, and container-based auto-scaling deployments.

Keywords: Cloud Computing, Amazon Web Services, E-Commerce, EC2, RDS, S3, IAM, VPC, PHP, MySQL, Web Application Deployment.

LIST OF FIGURES

Figure No.	Title	Page No.
10.1	AWS Architecture Diagram	—
10.2	Component Diagram	—
10.3	Deployment Diagram	—
10.4	Data Flow Diagram – Level 0	—
10.5	Data Flow Diagram – Level 1	—
11.1	Entity Relationship Diagram	—
13.1	Activity Diagram – User Workflow	—
13.2	Sequence Diagram – Checkout Process	—
18.1 – 18.15	Application and AWS Console Screenshots	—

LIST OF TABLES

Table No.	Title	Page No.
8.1	Functional Requirements	—
8.2	Software Requirements	—
8.3	Hardware Requirements	—
11.1	users Table Structure	—
11.2	products Table Structure	—

Table No.	Title	Page No.
11.3	cart Table Structure	—
11.4	wishlist Table Structure	—
11.5	orders Table Structure	—
11.6	order_items Table Structure	—
17.1	Software Test Cases	—

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AWS	Amazon Web Services
EC2	Elastic Compute Cloud
RDS	Relational Database Service
S3	Simple Storage Service
IAM	Identity and Access Management
VPC	Virtual Private Cloud
SG	Security Group
DB	Database
SQL	Structured Query Language
PHP	Hypertext Preprocessor
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
API	Application Programming Interface
HTTP/HTTPS	HyperText Transfer Protocol (Secure)
DFD	Data Flow Diagram
ER Diagram	Entity Relationship Diagram
CRUD	Create, Read, Update, Delete
PK / FK	Primary Key / Foreign Key
CIDR	Classless Inter-Domain Routing
AMI	Amazon Machine Image
SSH	Secure Shell
PDO	PHP Data Objects
SSL/TLS	Secure Sockets Layer / Transport Layer Security
UI/UX	User Interface / User Experience
CI/CD	Continuous Integration / Continuous Deployment

1. INTRODUCTION

1.1 Introduction

Electronic commerce, commonly referred to as e-commerce, has fundamentally transformed the manner in which goods and services are marketed, purchased, and delivered across the globe. What began as a supplementary sales channel for large retailers has, over the past two decades, become a primary mode of commerce for businesses of every scale. The rapid proliferation of internet connectivity, smartphones, and digital payment systems has created an environment in which even small enterprises and individual entrepreneurs can establish an online storefront and reach customers far beyond their immediate geographic location.

Historically, deploying a functional e-commerce platform required significant upfront capital investment in physical servers, networking equipment, and dedicated data-center space, along with the technical staff needed to maintain this infrastructure. This barrier to entry often placed cloud-scale reliability and performance out of reach for students, startups, and small businesses. The emergence of **cloud computing**, and in particular **Infrastructure-as-a-Service (IaaS)** and **Platform-as-a-Service (PaaS)** offerings from providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform, has fundamentally altered this landscape by allowing developers to provision computing, storage, and networking resources on demand, paying only for what they consume.

CloudCart is conceived as an academic yet practically representative mini e-commerce platform that demonstrates the application of these cloud-native principles. The system allows registered users to browse a catalogue of products, search and filter items by category, maintain a wishlist of desired products, add items to a shopping cart, and complete purchases through a structured checkout and order-management workflow. Rather than being hosted on a single physical machine, CloudCart is deployed across multiple AWS managed services — Amazon EC2 for compute, Amazon RDS for relational data persistence, and Amazon S3 for scalable object storage — each selected for its specific strength in the overall system architecture.

This project was undertaken with the dual objective of (a) building a functionally complete e-commerce web application using standard web development technologies, and (b) gaining hands-on, practical experience in provisioning, securing, and managing a multi-tier application on a leading public cloud platform. The result is a system that closely mirrors the architectural patterns used by real-world production e-commerce platforms, scaled appropriately for an academic final-year project.

1.2 Background

The retail industry has undergone three broad waves of transformation: the shift from purely physical storefronts to catalogue and telephone-based retail, the emergence of desktop-based online shopping in the late 1990s and early 2000s, and the current era of mobile-first, cloud-hosted, highly personalized commerce. Each wave has been enabled by

advances in underlying technology — the internet itself, secure payment processing, and now, elastic cloud infrastructure.

Traditional web hosting models, whether based on shared hosting, dedicated servers, or privately owned data centers, impose fixed capacity limits. An e-commerce site hosted on a single physical server may perform adequately under normal traffic conditions but can fail catastrophically during demand spikes such as festive sales or promotional events. Furthermore, such deployments typically bundle the web server, application logic, and database on a single machine, creating a single point of failure and making the system difficult to scale, secure, or maintain independently at each tier.

Cloud computing addresses these limitations by decoupling compute, storage, and database services into independently scalable, managed components. AWS, as the market-leading cloud provider, offers a mature and well-documented ecosystem of services that are widely used in both industry and academia to teach modern deployment practices. By building CloudCart on AWS, this project situates itself within the current industry-standard approach to deploying web applications, rather than relying on outdated single-server hosting paradigms.

1.3 Motivation

The motivation for this project stems from several converging factors:

1. **Industry Relevance:** Cloud computing skills, and AWS proficiency in particular, are among the most sought-after competencies in the software industry today. Employers increasingly expect graduating engineers to have practical exposure to cloud deployment, not merely theoretical knowledge of cloud concepts.
2. **Bridging Theory and Practice:** Academic coursework often covers web development and database management as isolated subjects. This project provides an opportunity to integrate these disciplines into a single, cohesive, deployable system, while also incorporating cloud infrastructure concepts that are typically underrepresented in traditional curricula.
3. **Cost-Effective Learning Environment:** AWS offers a Free Tier that enables students to provision EC2 instances, RDS databases, and S3 buckets without incurring significant costs, making it an ideal platform for academic experimentation with production-grade infrastructure.
4. **Real-World Problem Simulation:** E-commerce is a domain that most students intuitively understand as consumers, which makes it an effective vehicle for demonstrating complex backend concepts such as session management, transactional order processing, and data normalization in a context that is immediately relatable.
5. **Security Awareness:** Building and deploying a system that handles user credentials, personal information, and transactional data provides practical grounding in secure coding practices, such as password hashing and prepared

statements, as well as cloud-specific security controls such as IAM policies and Security Groups.

1.4 Scope

The scope of the CloudCart project encompasses the following:

- Design and development of a responsive frontend using HTML5, CSS3, and JavaScript that allows users to interact with the product catalogue, cart, and wishlist without requiring a full page reload for common actions.
- Development of a PHP-based backend that exposes functionality for user authentication, product management, cart operations, wishlist operations, checkout processing, and order retrieval.
- Design of a normalized MySQL relational database schema comprising six core tables: users, products, cart, wishlist, orders, and order_items.
- Deployment of the complete application stack on AWS, using EC2 for application hosting, RDS for the managed MySQL database, and S3 for product image storage.
- Implementation of network security controls using a custom VPC with public and private subnets, and Security Groups restricting inbound and outbound traffic to only the necessary ports and protocols.
- Implementation of access control using IAM roles that grant the EC2 instance limited, scoped permissions to interact with S3, following the principle of least privilege.
- Functional testing of all modules to validate correctness, along with basic performance and security testing.

The scope of this project is explicitly limited to a mini e-commerce platform suitable for academic demonstration. It does not include integration with a live third-party payment gateway, multi-vendor marketplace functionality, or a mobile native application, though these are discussed as potential future enhancements in Chapter 21.

1.5 Need of the Project

The need for a project of this nature arises from a clear gap between the theoretical instruction typically provided in academic programs and the practical, hands-on skills demanded by the software industry. Many students complete their coursework with a strong grasp of programming languages and database theory but limited exposure to how these components are assembled into a deployed, internet-facing system.

CloudCart addresses this need by requiring the project team to engage with the complete software development lifecycle — from requirement gathering and system design through implementation, cloud deployment, and testing. It further requires the team to reason about non-functional concerns such as scalability, availability, and security, which are often absent from purely academic exercises. In doing so, the project serves both as a demonstrable portfolio artifact and as a structured learning exercise in modern, cloud-native software engineering practice.

1.6 Existing Problems

Small and medium-scale businesses, as well as individual entrepreneurs seeking to establish an online retail presence, frequently encounter the following problems when relying on traditional, non-cloud-based hosting solutions:

1. **High Initial Infrastructure Cost:** Procuring and maintaining physical servers, networking hardware, and backup systems requires substantial upfront capital that is often unavailable to small businesses and students.
2. **Poor Scalability:** A fixed-capacity server cannot automatically respond to sudden increases in traffic, such as those experienced during promotional sales, leading to slow response times or complete service outages.
3. **Single Point of Failure:** When the web server, application code, and database reside on a single physical machine, any hardware failure results in complete downtime of the platform, with no built-in redundancy.
4. **Weak Security Posture:** Self-managed servers require the operator to independently manage operating system patches, firewall configuration, and intrusion detection, tasks that are frequently neglected due to lack of expertise or time, leaving the system vulnerable to attack.
5. **Manual Backup and Recovery:** Traditional hosting typically requires manually scripted backup routines for the database and application files, with recovery processes that are slow and error-prone.
6. **Limited Geographic Reach:** A single-location server introduces latency for users located far from the hosting data center, degrading the user experience for a geographically distributed customer base.
7. **Maintenance Overhead:** Operating system updates, security patching, and hardware maintenance consume significant time and expertise that small teams may not possess.

1.7 Challenges

Building CloudCart to address the above problems introduced its own set of technical and operational challenges that the project team had to overcome:

- **Selecting the Appropriate AWS Services:** With a wide array of AWS services available, identifying the minimal, most appropriate set of services (EC2, RDS, S3, IAM, VPC, Security Groups) required careful evaluation against the project's functional and cost requirements.
- **Network Configuration:** Designing a VPC with correctly segmented public and private subnets, route tables, and Security Group rules that permit legitimate traffic while blocking unauthorized access demanded a solid understanding of cloud networking fundamentals.

- **Secure Credential Management:** Ensuring that database credentials, session tokens, and AWS access keys were never hardcoded or exposed in client-facing code required disciplined secure coding practices.
- **Database Design under Normalization Constraints:** Designing a schema that avoided data redundancy while still supporting efficient queries for cart, wishlist, and order operations required multiple iterations of schema refinement.
- **Cross-Origin and Session Management:** Coordinating session state between the JavaScript frontend and the PHP backend, particularly for cart and wishlist persistence across page interactions, required careful handling of PHP sessions and AJAX request design.
- **Cost Management within Free Tier Limits:** Configuring EC2 and RDS instance types to remain within AWS Free Tier eligibility while still delivering acceptable performance required attention to instance sizing.

1.8 Need for Cloud-Based E-Commerce

Cloud-based deployment directly addresses each of the problems identified in Section 2.1:

- **Elastic Compute:** Amazon EC2 allows the compute capacity allocated to the web server to be resized or replicated without requiring new physical hardware, providing a pathway to horizontal or vertical scaling as demand grows.
- **Managed Database Services:** Amazon RDS abstracts away database server maintenance, including automated backups, patching, and failover configuration, allowing the development team to focus on schema design and query optimization rather than database administration.
- **Durable Object Storage:** Amazon S3 provides virtually unlimited, highly durable storage for product images, decoupling static asset storage from the application server and improving both reliability and content delivery performance.
- **Granular Access Control:** AWS IAM enables fine-grained, auditable permission policies that ensure each component of the system has access only to the resources it strictly requires.
- **Network Isolation:** Amazon VPC and Security Groups allow the system architect to define precise network boundaries, isolating the database tier from direct public internet access while still permitting the application tier to reach it securely.
- **Pay-as-you-go Economics:** Cloud pricing models eliminate large upfront capital expenditure, replacing it with operational expenditure proportional to actual resource consumption, which is particularly advantageous for academic projects and early-stage businesses.

These characteristics collectively establish a compelling case for the adoption of cloud infrastructure in the design and deployment of even small-scale e-commerce systems such as CloudCart, and form the foundation for the proposed system described in Chapter 4.

1.9 Traditional E-Commerce Deployment

In a conventional, non-cloud deployment model, an e-commerce application is typically hosted using one of the following approaches:

1. **Shared Hosting:** Multiple websites, potentially belonging to unrelated organizations, share the resources of a single physical server managed by a hosting provider. Each tenant is allocated a limited quota of CPU, memory, and storage.
2. **Dedicated Server Hosting:** An organization leases or purchases a physical server dedicated exclusively to its application, hosted either on-premises or in a third-party data center.
3. **Self-Managed Virtual Private Server (VPS):** A virtual machine is provisioned on shared physical hardware, offering more isolation than shared hosting but still requiring the tenant to manage the operating system, web server, and database independently.

In each of these models, the web server (commonly Apache or Nginx), the application runtime (such as PHP), and the database engine (such as MySQL) are frequently co-located on the same machine or a small, statically provisioned cluster. Scaling, when necessary, typically involves manually provisioning additional servers and configuring load balancing, a process that can take hours or days rather than minutes.

1.10 Limitations of the Existing System

The traditional deployment model described above exhibits the following specific limitations when applied to an e-commerce use case:

- **Static Capacity Planning:** System administrators must estimate peak traffic in advance and provision hardware accordingly. Under-provisioning leads to slow performance or outages during high-demand periods, while over-provisioning results in wasted expenditure on idle capacity.
- **Tight Coupling of Application Tiers:** Co-locating the web server and database on a single machine means that a resource-intensive database query can degrade the performance of the web server, and vice versa, with no isolation between tiers.
- **Manual Disaster Recovery:** Recovering from hardware failure typically requires manual reconstruction of the server environment from backups, a process that can result in extended downtime and potential data loss if backups are not current.
- **Limited Automation:** Traditional hosting environments often lack the tooling for automated scaling, health monitoring, and self-healing that are standard features of managed cloud services.

1.11 Security Concerns in the Existing System

Security in traditional deployments is entirely the responsibility of the hosting tenant or a third-party hosting provider with limited configurability:

- Operating system and software patches must be applied manually, and delays in patching create windows of vulnerability to known exploits.
- Firewall rules are often configured using generic, host-based tools that lack the auditability and version control of cloud-native security constructs.
- Database servers are frequently exposed on the same network segment as the web server, increasing the attack surface if the web tier is compromised.
- There is typically no equivalent of IAM-style, resource-scoped access policies; access control is often limited to operating-system user accounts and file permissions.

1.12 Cost Issues in the Existing System

Traditional infrastructure requires significant capital expenditure (CAPEX) for hardware procurement, in addition to ongoing operational expenditure (OPEX) for power, cooling, physical security, and skilled personnel to maintain the environment. Because capacity must be provisioned for anticipated peak load rather than actual average load, a substantial proportion of this investment remains underutilized during normal operating periods, resulting in poor overall cost efficiency.

1.13 Scalability Issues in the Existing System

Scaling a traditionally hosted e-commerce platform, whether vertically (adding more resources to an existing server) or horizontally (adding more servers), is a manual, time-consuming process that typically requires:

- Procuring or provisioning new hardware.
- Manually installing and configuring the operating system, web server, and application dependencies.
- Reconfiguring load balancers and DNS records to incorporate the new capacity.
- Extensive testing to ensure the newly added capacity integrates correctly with the existing environment.

This lack of elasticity means that traditional systems are poorly suited to handle sudden, unpredictable traffic spikes, a common occurrence in e-commerce during promotional events, seasonal sales, or viral marketing campaigns — precisely the scenarios in which system availability matters most.

1.14 Overview

The proposed system, CloudCart, addresses the limitations of traditional e-commerce deployment identified in Chapter 3 by re-architecting the application around AWS managed cloud services. Rather than co-locating the web server, application logic, and database on a single physical machine, CloudCart decomposes the system into independently provisioned and independently scalable tiers, each backed by a purpose-built AWS service.

1.15 Architecture of the Proposed System

The high-level architecture of CloudCart consists of four principal AWS components working in concert:

- **Amazon EC2** hosts the Apache web server and PHP runtime, serving both the static frontend assets and the dynamic PHP backend endpoints.
- **Amazon RDS (MySQL)** provides a fully managed relational database instance, decoupled from the application server, to persist user, product, cart, wishlist, and order data.
- **Amazon S3** stores and serves product images as durable, scalable object storage, independent of the EC2 instance's local disk.
- **Amazon VPC, IAM, and Security Groups** collectively provide the networking and access-control fabric that isolates and secures the above components.

This architecture is illustrated in detail in Chapter 10 (System Architecture).

1.16 Cloud-Based Deployment Model

CloudCart is deployed using the following provisioning strategy:

1. A VPC is created with a public subnet (hosting the EC2 instance) and a private subnet (hosting the RDS instance), ensuring that the database is never directly reachable from the public internet.
2. The EC2 instance is launched using an Amazon Linux Amazon Machine Image (AMI), with Apache and PHP installed and configured to serve the CloudCart application.
3. An RDS MySQL instance is provisioned within the private subnet, with its Security Group configured to accept inbound connections only from the EC2 instance's Security Group on port 3306.
4. An S3 bucket is created to store product images, with an IAM role attached to the EC2 instance granting it scoped PutObject and GetObject permissions on that specific bucket.
5. Security Groups are configured at each tier to permit only the minimal necessary inbound and outbound traffic, following the principle of least privilege.

1.17 Advantages of the Proposed System

- **Elastic Scalability:** The EC2 instance type can be resized, or additional instances can be added behind a load balancer, without any change to the underlying database or storage tiers.
- **Managed Database Administration:** RDS automates routine database administration tasks including backups, minor version patching, and storage scaling, reducing operational burden.
- **Durable, Independent Storage:** S3 offers eleven nines of designed durability for stored objects, ensuring product images remain available and are served independently of the application server's health.

- **Improved Security Posture:** The combination of VPC subnet isolation, Security Groups, and IAM policies creates multiple layers of defense, significantly reducing the system's attack surface compared to the traditional model.
- **Reduced Operational Cost:** Pay-as-you-go pricing, combined with AWS Free Tier eligibility for development and testing, minimizes the cost of building and demonstrating the system.
- **High Availability Potential:** Although the academic deployment uses a single EC2 instance and a single-AZ RDS instance for cost reasons, the architecture can be readily extended to Multi-AZ RDS deployments and Auto Scaling Groups for production-grade availability.

1.18 Benefits Summary

Aspect	Traditional System	Proposed CloudCart System
Infrastructure Provisioning	Manual, hardware-based	On-demand, software-defined
Scalability	Static, slow to scale	Elastic, can scale in minutes
Database Management	Manual administration	Managed by Amazon RDS
Storage for Media Assets	Local disk on web server	Amazon S3 (independent, durable)
Network Security	Host-based firewall only	VPC + Security Groups + IAM
Cost Model	High CAPEX, fixed OPEX	Pay-as-you-go OPEX
Disaster Recovery	Manual backup/restore	Automated RDS snapshots

1.19 Objectives

The CloudCart project is guided by the following specific, measurable objectives:

1. **To design and develop a functional mini e-commerce web application** that supports user registration, authentication, product browsing, search, category filtering, wishlist management, cart management, checkout, and order tracking.
2. **To design a normalized relational database schema** in MySQL capable of efficiently modelling the relationships between users, products, cart items, wishlist items, orders, and order line items, while avoiding data redundancy and update anomalies.
3. **To deploy the complete application stack on Amazon Web Services**, utilizing Amazon EC2 for compute hosting, Amazon RDS for managed relational database services, and Amazon S3 for product image storage.
4. **To implement a secure network architecture** using Amazon VPC with properly segmented public and private subnets, ensuring that the database tier is not directly exposed to the public internet.

5. **To configure fine-grained access control** using AWS IAM roles and policies, granting each system component only the permissions strictly necessary for its function, in accordance with the principle of least privilege.
6. **To implement Security Group rules** at each network tier that restrict inbound and outbound traffic to the minimal set of ports and protocols required for correct system operation.
7. **To apply secure coding practices** in the PHP backend, including password hashing using industry-standard algorithms, prepared statements to prevent SQL injection, and secure session management to prevent session hijacking.
8. **To validate system correctness through structured testing**, including unit testing of individual PHP functions, integration testing of the frontend-backend-database pipeline, and functional testing of each user-facing module.
9. **To document the complete system design, deployment process, and testing outcomes** in a manner suitable for academic evaluation and for future reference by developers extending the system.
10. **To evaluate the cost, scalability, and security benefits** of the proposed cloud-based deployment relative to traditional, non-cloud hosting approaches, and to identify realistic avenues for future enhancement.

1.20 Introduction to the Literature Survey

This chapter reviews existing academic literature, technical documentation, and industry publications relevant to the design and deployment of cloud-hosted e-commerce systems. The survey spans four broad areas: (i) foundational cloud computing concepts, (ii) Amazon Web Services architecture and best practices, (iii) web application development technologies, and (iv) e-commerce system design. This body of literature informed the architectural and implementation decisions made throughout the CloudCart project.

1.21 Cloud Computing Fundamentals

Mell and Grance [1], in the widely cited National Institute of Standards and Technology (NIST) definition, characterize cloud computing through five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service. These characteristics directly motivate the architectural choices made in CloudCart, particularly the use of on-demand, independently scalable EC2 and RDS resources rather than statically provisioned hardware.

Armbrust et al. [2] provide an early and influential analysis of the economic and technical trade-offs of cloud computing, arguing that the elasticity and pay-as-you-go pricing of cloud platforms are particularly advantageous for applications with variable or unpredictable demand, a characteristic strongly applicable to e-commerce platforms subject to seasonal and promotional traffic spikes.

Buyya et al. [3] discuss the classification of cloud services into Infrastructure-as-a-Service, Platform-as-a-Service, and Software-as-a-Service models, providing the conceptual foundation for understanding how EC2 (IaaS), RDS (a managed PaaS-like database service), and S3 (a storage-focused managed service) fit within the broader cloud service taxonomy.

1.22 Amazon Web Services Architecture and Best Practices

The AWS Well-Architected Framework [4] establishes five (subsequently six) pillars of cloud architecture: operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. This framework guided the design principles adopted in CloudCart, particularly with respect to the security pillar (VPC segmentation, IAM least privilege, Security Groups) and the cost optimization pillar (Free Tier-eligible instance sizing).

The official AWS documentation for Amazon EC2 [5] describes the elastic compute service as a resizable compute capacity in the cloud, supporting a wide range of instance types optimized for different workloads. This project utilizes a general-purpose `t2.micro` or `t3.micro` instance type, consistent with AWS Free Tier eligibility guidelines.

The AWS documentation for Amazon RDS [6] details the managed relational database service's support for automated backups, software patching, and simplified administration compared to self-managed database servers, directly informing the decision to use RDS rather than installing MySQL manually on the EC2 instance.

The AWS documentation for Amazon S3 [7] describes the object storage service's eleven nines (99.99999999%) of designed durability, motivating its selection for storing product images independently of the application server's ephemeral local storage.

Varia and Mathew [8], in an AWS whitepaper on cloud architectures, discuss reference patterns for multi-tier web applications, including the separation of web, application, and database tiers across distinct subnets — a pattern directly reflected in the CloudCart VPC design described in Chapter 10.

The AWS Identity and Access Management documentation [9] emphasizes the principle of least privilege as a foundational security best practice, recommending that IAM roles be scoped to the minimum set of permissions required for a given task, which directly informed the IAM role configuration for the EC2-to-S3 access pattern used in this project.

The AWS VPC documentation [10] describes the use of route tables, subnets, and internet gateways to construct isolated virtual networks within AWS, forming the technical basis for the public/private subnet segregation implemented in CloudCart.

1.23 Web Application Development Technologies

Duckett [11] provides a comprehensive treatment of HTML5 and CSS3 as the foundational markup and styling languages for modern web interfaces, emphasizing semantic markup and responsive design principles that were applied in the CloudCart frontend.

Flanagan [12] offers an authoritative reference on JavaScript (ECMAScript), covering asynchronous programming patterns such as the Fetch API, which CloudCart uses extensively to perform AJAX-style communication between the frontend and the PHP backend without full page reloads.

Lockhart and Lerdorf [13] document the PHP scripting language's suitability for server-side web development, particularly its native integration with MySQL via the PDO (PHP Data Objects) extension, which CloudCart uses for all database interactions to benefit from prepared-statement support and reduced SQL injection risk.

The MySQL Reference Manual [14] provides comprehensive guidance on relational schema design, indexing strategies, and normalization, informing the design of the CloudCart database schema described in Chapter 11.

OWASP [15] publishes widely referenced guidelines on secure web application development, including the OWASP Top Ten list of web application security risks. The CloudCart implementation directly addresses several OWASP Top Ten categories, including Injection (via prepared statements), Broken Authentication (via password hashing and session management), and Security Misconfiguration (via VPC and Security Group hardening).

1.24 E-Commerce System Design

Laudon and Traver [16] provide a foundational text on e-commerce business and technology, describing core functional requirements common to online retail platforms, including product catalog management, shopping cart persistence, and order fulfillment workflows, all of which are reflected in the CloudCart module design described in Chapter 12.

1.25 Summary of Literature Survey

The reviewed literature collectively establishes that (a) cloud computing offers measurable advantages in scalability, cost, and operational efficiency over traditional hosting models, (b) AWS provides a mature, well-documented set of services that align closely with the architectural requirements of multi-tier web applications, (c) established web development technologies (HTML5, CSS3, JavaScript, PHP, MySQL) remain well-suited for building functionally complete e-commerce systems, and (d) security best practices such as least-privilege IAM policies, network segmentation, and secure coding practices are essential, well-documented components of any production-representative deployment. These findings directly informed the design and implementation decisions detailed in the subsequent chapters of this report.

A feasibility study was conducted prior to the commencement of development to evaluate whether the proposed CloudCart system could be realistically designed, implemented, and deployed within the constraints of an academic project. The study considered five dimensions of feasibility.

1.26 Technical Feasibility

The technologies selected for CloudCart — HTML5, CSS3, JavaScript, PHP, MySQL, and the AWS services EC2, RDS, S3, IAM, and VPC — are all mature, well-documented, and widely adopted in both industry and academic contexts. Comprehensive official documentation, active developer communities, and extensive third-party tutorials exist for each technology, substantially reducing the technical risk associated with the project. Team members possessed foundational knowledge of web development and relational databases from prior coursework, and the AWS Free Tier provided a no-cost environment in which to acquire the necessary cloud deployment skills. The project was therefore assessed as technically feasible within the available timeframe.

1.27 Economic Feasibility

The financial cost of developing and deploying CloudCart was assessed as minimal. AWS Free Tier eligibility allows for 750 hours per month of a t2.micro or t3.micro EC2 instance, 750 hours per month of a db.t3.micro RDS instance, and 5 GB of S3 storage, all free of charge for twelve months from account creation. These allowances comfortably cover the compute, database, and storage requirements of an academic project of this scale. No licensing costs were incurred, as all software components (Apache, PHP, MySQL client libraries, and development tools) are open-source or freely available. The project was therefore assessed as economically feasible with negligible financial outlay.

1.28 Operational Feasibility

The proposed system was designed to be operable by the project team without requiring specialized infrastructure engineering expertise beyond what could be acquired through AWS documentation and standard system administration practices. The AWS Management Console provides a graphical interface for provisioning and monitoring all required services, reducing the operational complexity relative to self-managed, on-premises infrastructure. The modular separation of concerns (frontend, backend, database, and cloud infrastructure) also allowed team members to work in parallel on different components, improving overall development efficiency.

1.29 Legal Feasibility

CloudCart, as an academic demonstration project, does not process real financial transactions, real customer payment information, or genuine personally identifiable information beyond what is voluntarily provided by test users for demonstration purposes. The project does not, therefore, raise significant legal or regulatory concerns related to data protection or payment card industry compliance. Should the system be extended toward a production deployment handling real user data, compliance with applicable data protection regulations (such as the Information Technology Act, 2000, in the Indian context, and relevant AWS shared-responsibility model obligations) would need to be formally assessed, as discussed further in Chapter 20 (Limitations).

1.30 Schedule Feasibility

The project was planned and executed across a structured timeline spanning requirement analysis, system design, frontend and backend development, database design, AWS infrastructure provisioning, integration, and testing. The modular nature of the system, combined with the well-documented nature of the selected technologies, allowed the project to be completed within the academic semester timeframe allocated for the final-year major project, with sufficient buffer for testing and documentation.

Feasibility Dimension	Assessment	Justification
Technical	Feasible	Mature, well-documented technology stack
Economic	Feasible	AWS Free Tier covers all development and demonstration costs
Operational	Feasible	Console-based provisioning; modular development workflow
Legal	Feasible	No real financial or sensitive data processed
Schedule	Feasible	Achievable within one academic semester

2. METHODOLOGY

2.1 Functional Requirements

Table 8.1 summarizes the functional requirements identified for the CloudCart system.

Table 8.1: Functional Requirements

Req. ID	Requirement Description
FR-01	The system shall allow a new user to register an account by providing full name, email address, password, and phone number.
FR-02	The system shall allow a registered user to log in using their email address and password.
FR-03	The system shall allow a logged-in user to securely log out, terminating their active session.
FR-04	The system shall display a catalogue of available products with name, price, category, image, and stock availability.
FR-05	The system shall allow a user to search for products by keyword.
FR-06	The system shall allow a user to filter products by category.
FR-07	The system shall allow a logged-in user to add or remove products from a personal wishlist.

Req. ID	Requirement Description
FR-08	The system shall allow a logged-in user to add, update the quantity of, or remove products from a shopping cart.
FR-09	The system shall allow a logged-in user to proceed through a checkout process, providing a shipping address and confirming the order.
FR-10	The system shall calculate the total order amount based on the quantities and prices of cart items at the time of checkout.
FR-11	The system shall create an order record and associated order line items upon successful checkout, and shall clear the user's cart.
FR-12	The system shall allow a logged-in user to view their past order history.
FR-13	The system shall persist product images in Amazon S3 and retrieve them for display in the product catalogue.
FR-14	The system shall validate user input on both the client side (JavaScript) and server side (PHP) to prevent malformed data submission.

2.2 Non-Functional Requirements

- **Performance:** The system shall return product listing and search results within an acceptable response time (target: under 2 seconds) under normal load on a t2.micro / t3.micro EC2 instance.
- **Security:** The system shall store passwords using a one-way cryptographic hash, shall use parameterized queries for all database interactions, and shall restrict database access to the application tier only.
- **Availability:** The system shall remain accessible during normal operating hours, with the underlying AWS infrastructure configured to allow future extension to Multi-AZ RDS deployments for improved availability.
- **Usability:** The user interface shall be intuitive and require no specialized training for a typical online shopper to complete a purchase.
- **Scalability:** The architecture shall permit the EC2 compute tier to be scaled independently of the RDS database tier and the S3 storage tier.
- **Maintainability:** The codebase shall be organized into clearly separated frontend and backend modules, each with a single, well-defined responsibility, to facilitate future maintenance and extension.
- **Portability:** The application shall be deployable on any AWS region supporting EC2, RDS, and S3, without dependency on region-specific configuration beyond standard endpoint URLs.

2.3 Software Requirements

Table 8.2: Software Requirements

Component	Specification
-----------	---------------

Component	Specification
Operating System (Server)	Amazon Linux 2 / Amazon Linux 2023
Web Server	Apache HTTP Server 2.4
Server-Side Language	PHP 8.1 or later
Database Engine	MySQL 8.0 (via Amazon RDS)
Frontend Languages	HTML5, CSS3, JavaScript (ES6)
Cloud Platform	Amazon Web Services
Development Tools	Visual Studio Code, phpMyAdmin / MySQL Workbench, Git
Browser (Client)	Google Chrome, Mozilla Firefox, or equivalent modern browser

2.4 Hardware Requirements

Table 8.3: Hardware Requirements

Component	Specification
EC2 Instance Type	t2.micro / t3.micro (1 vCPU, 1 GB RAM) — AWS Free Tier eligible
RDS Instance Type	db.t3.micro (1 vCPU, 1 GB RAM) — AWS Free Tier eligible
Storage (EC2)	8–30 GB General Purpose SSD (gp2/gp3)
Storage (S3)	Standard storage class, pay-per-use, up to 5 GB under Free Tier
Client Device	Any device with a modern web browser and internet connectivity
Network	Broadband internet connection (minimum 1 Mbps recommended)

2.5 Overview

This chapter describes the complete user workflow within CloudCart, from initial registration through to order completion, using an Activity Diagram to capture the overall flow of control and a Sequence Diagram to capture the detailed interaction between system components during checkout.

2.6 Activity Diagram – User Workflow

Figure 13.1 illustrates the complete activity flow a typical customer follows when using CloudCart, from first visiting the homepage through registration/login, product discovery, wishlist and cart interactions, and finally checkout and order confirmation.

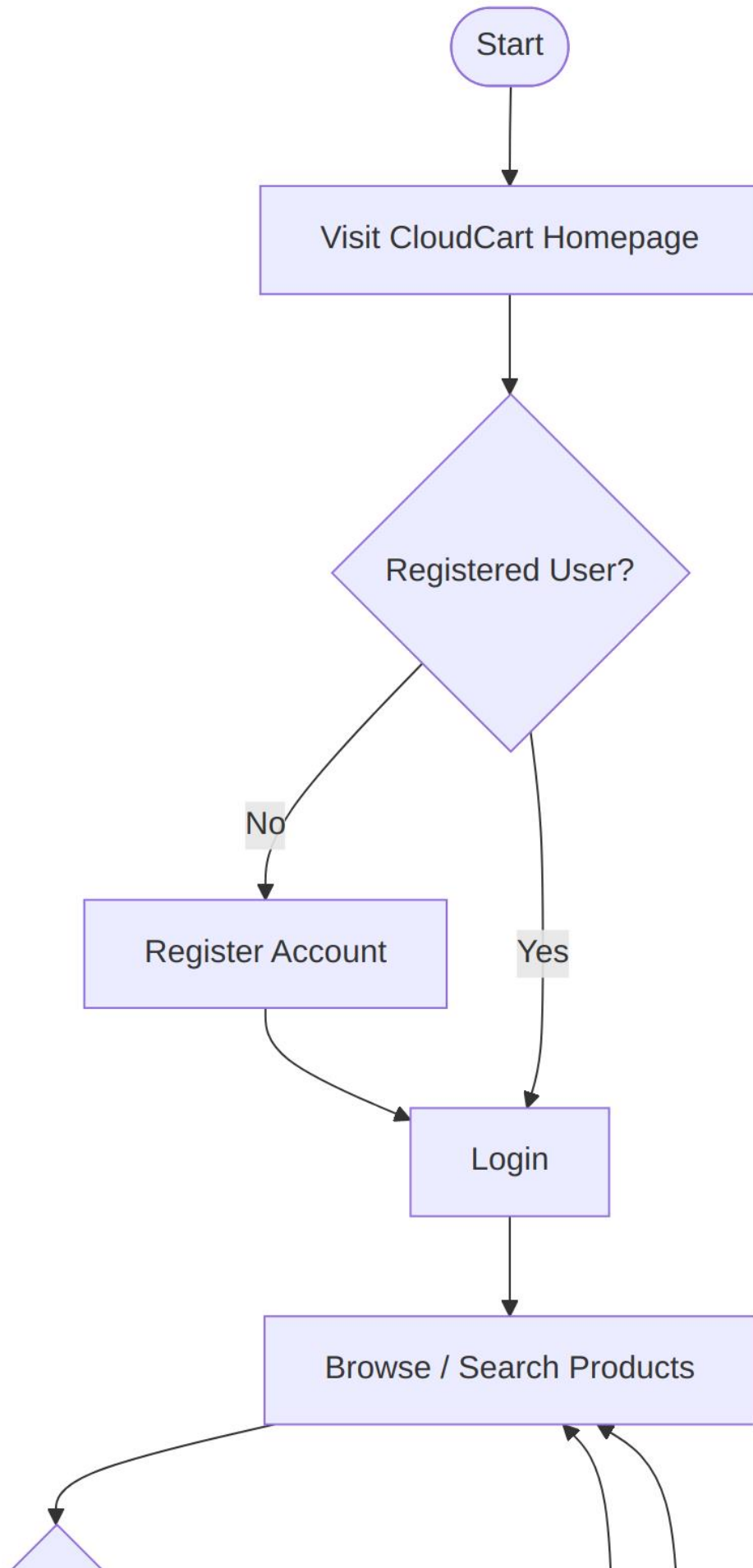


Figure 13.1: Activity Diagram - User Workflow

Figure 13.1: Activity Diagram - User Workflow

Figure 13.1 – Activity Diagram (User Workflow)

2.7 Sequence Diagram – Checkout Process

Figure 13.2 details the sequence of interactions between the user, the browser (frontend), the EC2-hosted PHP backend, and the RDS MySQL database during the checkout process, including the transactional creation of the order and order line items and the clearing of the cart.

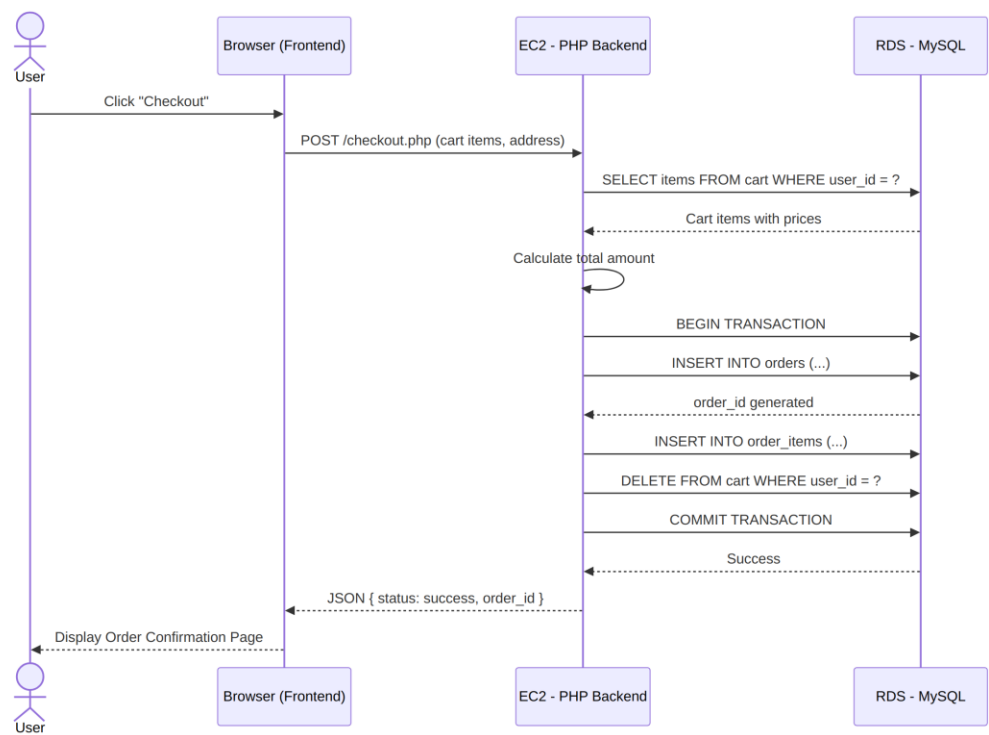


Figure 13.2: Sequence Diagram - Checkout Process

Figure 13.2: Sequence Diagram - Checkout Process

Figure 13.2 – Sequence Diagram (Checkout Process)

2.8 Workflow Narrative

1. A new visitor arrives at the CloudCart homepage and browses the product catalogue without needing to authenticate.
2. To perform actions such as adding items to a wishlist or cart, the visitor is prompted to register or log in.
3. Upon successful registration, the user's credentials are securely stored (with the password hashed) and the user is directed to log in.

4. Upon login, a session is established, and the user is redirected to the homepage with their authenticated state reflected in the navigation bar.
5. The user browses products, optionally narrowing the catalogue using the search bar or category filter.
6. The user may add individual products to their wishlist for later consideration, or directly to their cart for purchase.
7. The user reviews the cart contents, adjusting quantities or removing items as needed, before proceeding to checkout.
8. During checkout, the user provides or confirms a shipping address and submits the order.
9. The backend processes the order transactionally, ensuring that the order and its line items are created atomically and that the cart is cleared only upon successful order creation.
10. The user receives an order confirmation and may subsequently view the order in their order history.

This chapter presents pseudocode representations of the core algorithms implemented within the CloudCart backend.

2.9 Authentication Workflow

ALGORITHM: UserLogin

INPUT: email, password

OUTPUT: session token or error

1. START session handler
2. RECEIVE email, password from POST request
3. IF email or password is empty THEN
 RETURN error "Missing credentials"
4. QUERY users WHERE email = ? (prepared statement)
5. IF no matching record THEN
 RETURN error "Invalid credentials"
6. RETRIEVE stored password_hash from record
7. IF password_verify(password, password_hash) is FALSE THEN
 RETURN error "Invalid credentials"
8. SET \$_SESSION['user_id'] = user.user_id
9. SET \$_SESSION['full_name'] = user.full_name
10. RETURN success with user_id
11. END

2.10 Cart Management Algorithm

ALGORITHM: AddToCart

INPUT: user_id (from session), product_id, quantity

OUTPUT: updated cart summary

1. VERIFY active session; IF absent THEN RETURN error 401
2. VALIDATE product_id exists AND stock_quantity >= quantity
3. QUERY cart WHERE user_id = ? AND product_id = ?

4. IF record exists THEN
 UPDATE cart SET quantity = quantity + new_quantity
ELSE
 INSERT INTO cart (user_id, product_id, quantity)
5. RECALCULATE cart subtotal = SUM(quantity * price) for all cart rows of user
6. RETURN updated cart items and subtotal
7. END

2.11 Search Algorithm

ALGORITHM: SearchProducts

INPUT: keyword

OUTPUT: matching product list

1. RECEIVE keyword from query parameter
2. SANITIZE keyword (trim whitespace)
3. CONSTRUCT parameterized query:
 SELECT * FROM products
 WHERE product_name LIKE CONCAT('%', ?, '%')
 OR description LIKE CONCAT('%', ?, '%')
4. EXECUTE query with bound keyword parameter (prepared statement)
5. FORMAT result rows as JSON array
6. RETURN JSON array to client
7. END

2.12 Wishlist Algorithm

ALGORITHM: ToggleWishlist

INPUT: user_id (from session), product_id

OUTPUT: updated wishlist status

1. VERIFY active session; IF absent THEN RETURN error 401
2. QUERY wishlist WHERE user_id = ? AND product_id = ?
3. IF record exists THEN
 DELETE FROM wishlist WHERE user_id = ? AND product_id = ?
 SET status = "removed"
ELSE
 INSERT INTO wishlist (user_id, product_id)
 SET status = "added"
4. RETURN status
5. END

2.13 Checkout Process Algorithm

ALGORITHM: ProcessCheckout

INPUT: user_id (from session), shipping_address

OUTPUT: order confirmation or error

1. VERIFY active session; IF absent THEN RETURN error 401
2. BEGIN TRANSACTION
3. SELECT cart items JOIN products WHERE cart.user_id = ?

```

4. IF cart is empty THEN
    ROLLBACK; RETURN error "Cart is empty"
5. total_amount = SUM(cart.quantity * products.price)
6. INSERT INTO orders (user_id, total_amount, shipping_address, order_status)
    VALUES (user_id, total_amount, shipping_address, 'Pending')
7. order_id = LAST_INSERT_ID()
8. FOR EACH item IN cart items:
    INSERT INTO order_items (order_id, product_id, quantity, unit_price)
        VALUES (order_id, item.product_id, item.quantity, item.price)
    UPDATE products SET stock_quantity = stock_quantity - item.quantity
        WHERE product_id = item.product_id
9. DELETE FROM cart WHERE user_id = ?
10. COMMIT TRANSACTION
11. RETURN success with order_id
    ON ANY ERROR: ROLLBACK TRANSACTION; RETURN error
12. END

```

2.14 Session Handling Algorithm

ALGORITHM: ValidateSession

INPUT: implicit PHP session cookie

OUTPUT: authenticated user_id, or HTTP 401 error

```

1. CALL session_start()
2. IF $_SESSION['user_id'] is not set THEN
    SET HTTP response code 401
    RETURN JSON error "Unauthorized"
3. RETURN $_SESSION['user_id']
4. END

```

3. TECHNOLOGY STACK

This chapter describes, in detail, each of the technologies used to build and deploy CloudCart, including their definition, key features, advantages, specific role within the project, and the rationale for their selection over alternative technologies.

3.1 HTML5

Definition: HTML5 is the fifth and current major revision of the HyperText Markup Language, the standard markup language used to structure content on the World Wide Web.

Features: Semantic elements (<header>, <nav>, <section>, <article>, <footer>), native form validation attributes, improved multimedia support, and broad cross-browser compatibility.

Advantages: Improves accessibility and search-engine optimization through semantic structure; reduces reliance on third-party plugins; is natively supported by all modern browsers without additional dependencies.

Role in CloudCart: HTML5 forms the structural backbone of the `index.html` file, defining the layout of the product catalogue, navigation bar, cart panel, wishlist panel, and checkout forms.

Why Selected: As the universally supported foundation of web content, HTML5 was the natural choice for structuring the CloudCart frontend without introducing unnecessary framework overhead, keeping the application lightweight and fast-loading.

3.2 CSS3

Definition: CSS3 is the latest evolution of Cascading Style Sheets, the language used to describe the visual presentation of HTML documents.

Features: Flexbox and Grid layout systems, transitions and animations, media queries for responsive design, and custom properties (CSS variables).

Advantages: Enables a clean separation between content (HTML) and presentation (CSS); supports responsive design across desktop and mobile viewports without duplicating markup; improves maintainability through reusable style rules.

Role in CloudCart: CSS3, implemented in `style.css`, governs the visual styling of all pages, including the product grid layout, cart and wishlist panels, form styling, and responsive breakpoints for mobile compatibility.

Why Selected: CSS3's native Flexbox and Grid capabilities were sufficient to implement a clean, responsive product catalogue layout without requiring a heavier CSS framework, keeping page load times low.

3.3 JavaScript (ES6)

Definition: JavaScript is a high-level, interpreted scripting language that enables dynamic, interactive behaviour within web pages, executed natively within the browser.

Features: ES6 (ECMAScript 2015) and later introduced arrow functions, `let/const` block scoping, Promises, and the Fetch API for asynchronous HTTP communication.

Advantages: Enables asynchronous, non-blocking communication with the backend without full page reloads; runs natively in every modern browser without additional runtime installation; provides a rich ecosystem of DOM manipulation capabilities.

Role in CloudCart: JavaScript, implemented in `script.js`, handles client-side interactivity including dynamic product rendering, search-as-you-type filtering, AJAX calls (via the Fetch API) to the PHP backend for cart, wishlist, and checkout operations, and client-side form validation.

Why Selected: JavaScript was selected over a heavier frontend framework (such as React or Angular) to keep the application's dependency footprint minimal, consistent with the project's goal of demonstrating core web and cloud concepts without additional framework-specific complexity.

3.4 PHP

Definition: PHP (recursively, “PHP: Hypertext Preprocessor”) is a widely used open-source server-side scripting language specifically designed for web development.

Features: Native session management, extensive built-in string and array manipulation functions, and the PDO (PHP Data Objects) extension for consistent, secure database access across multiple database engines.

Advantages: Broad hosting compatibility and mature ecosystem; native session handling simplifies authentication state management; PDO prepared statements provide strong protection against SQL injection.

Role in CloudCart: PHP implements all server-side business logic, including `login.php`, `register.php`, `logout.php`, `products.php`, `cart.php`, `wishlist.php`, `checkout.php`, and `orders.php`, each exposing a focused set of operations to the frontend, with `db.php` centralizing the database connection logic.

Why Selected: PHP’s tight integration with Apache and MySQL, combined with its low barrier to entry and extensive documentation, made it well-suited for rapidly building a secure, functional backend within the academic project timeframe.

3.5 MySQL

Definition: MySQL is a widely used open-source relational database management system (RDBMS) based on Structured Query Language (SQL).

Features: ACID-compliant transactions (with the InnoDB storage engine), foreign key constraint enforcement, indexing for query optimization, and broad tool support (MySQL Workbench, phpMyAdmin).

Advantages: Mature, well-documented, and widely adopted, with native support from Amazon RDS; strong consistency guarantees suitable for transactional e-commerce operations such as order placement.

Role in CloudCart: MySQL, hosted via Amazon RDS, stores all persistent application data across the six core tables described in Chapter 11, including user accounts, product listings, cart and wishlist entries, and order records.

Why Selected: MySQL’s relational model is well-suited to the highly structured, relationally connected nature of e-commerce data (users, products, orders), and its native support as a managed engine within Amazon RDS eliminates the need for manual database server administration.

3.6 Apache HTTP Server

Definition: Apache HTTP Server is a widely deployed, open-source web server software responsible for handling HTTP requests and serving web content.

Features: Modular architecture supporting PHP integration via `mod_php` or PHP-FPM, `.htaccess`-based configuration, and robust logging capabilities.

Advantages: Highly stable and extensively documented; natively supported on Amazon Linux AMIs; straightforward integration with PHP for dynamic content generation.

Role in CloudCart: Apache runs on the EC2 instance, listening on port 80/443, routing incoming HTTP requests to the appropriate static HTML/CSS/JavaScript assets or PHP backend scripts.

Why Selected: Apache's maturity, extensive community support, and straightforward configuration on Amazon Linux made it the preferred choice over alternatives such as Nginx for this project's scope.

3.7 Amazon EC2

Definition: Amazon Elastic Compute Cloud (EC2) is a web service that provides resizable virtual compute capacity in the AWS cloud.

Features: A wide range of instance types and sizes, pay-per-use or reserved pricing models, integration with Elastic Load Balancing and Auto Scaling Groups, and Security Group-based network access control.

Advantages: Eliminates the need to procure physical servers; instances can be resized or replicated on demand; integrates natively with the broader AWS ecosystem.

Role in CloudCart: A `t2.micro/t3.micro` EC2 instance hosts the Apache web server and PHP runtime, serving as the compute tier of the application.

Why Selected: EC2 provides Free Tier-eligible, general-purpose compute capacity sufficient for hosting a mini e-commerce application, while offering a clear upgrade path to larger instance types or Auto Scaling Groups should demand increase.

3.8 Amazon RDS

Definition: Amazon Relational Database Service (RDS) is a managed service that simplifies the setup, operation, and scaling of relational databases in the cloud.

Features: Automated backups and point-in-time recovery, automated software patching, Multi-AZ deployment options for high availability, and support for multiple database engines including MySQL.

Advantages: Removes the operational burden of manual database administration; provides built-in high-availability and backup capabilities; decouples the database tier from the application server for independent scaling.

Role in CloudCart: A `db.t3.micro` RDS MySQL instance stores all persistent relational data for the application, isolated within a private subnet and accessible only from the EC2 instance's Security Group.

Why Selected: RDS was selected over self-managing MySQL on an EC2 instance to benefit from automated backups, patching, and the improved security posture of running the database in a private, non-internet-facing subnet.

3.9 Amazon S3

Definition: Amazon Simple Storage Service (S3) is an object storage service offering industry-leading scalability, data availability, security, and performance.

Features: Virtually unlimited storage capacity, eleven nines of designed durability, fine-grained access control via bucket policies and IAM, and support for static content hosting.

Advantages: Decouples static asset (image) storage from the application server's local disk; highly durable and available; cost-effective for storing large volumes of media files.

Role in CloudCart: An S3 bucket stores all product images uploaded through the product management workflow, referenced by URL from the products table and rendered in the frontend product catalogue.

Why Selected: S3 was selected over storing images on the EC2 instance's local Elastic Block Store (EBS) volume to ensure image persistence independent of the instance lifecycle, and to benefit from S3's scalability and durability guarantees.

3.10 AWS IAM

Definition: AWS Identity and Access Management (IAM) is a web service that enables secure control of access to AWS resources through users, groups, roles, and policies.

Features: Fine-grained, JSON-based permission policies; support for role-based access without long-lived credentials; integration with virtually every AWS service.

Advantages: Enables the principle of least privilege to be enforced precisely; avoids embedding long-lived AWS access keys within application code; provides a centralized, auditable access-control mechanism.

Role in CloudCart: An IAM role is attached to the EC2 instance, granting it scoped permissions to perform PutObject and GetObject operations on the specific S3 bucket used for product images, without granting broader account-level access.

Why Selected: IAM roles were selected over hardcoded AWS access keys to eliminate the security risk of credential leakage and to align with AWS-recommended best practices for EC2-to-S3 access.

3.11 Amazon VPC

Definition: Amazon Virtual Private Cloud (VPC) enables the provisioning of a logically isolated section of the AWS cloud in which AWS resources can be launched within a user-defined virtual network.

Features: Custom IP address ranges (CIDR blocks), public and private subnets, route tables, internet gateways, and network access control lists (NACLs).

Advantages: Provides complete control over network topology; enables strict isolation of sensitive resources (such as the database tier) from direct internet exposure; supports fine-grained routing control.

Role in CloudCart: A custom VPC hosts the EC2 instance in a public subnet (with a route to an internet gateway) and the RDS instance in a private subnet (with no direct route to the internet), ensuring the database is reachable only from within the VPC.

Why Selected: A custom VPC, rather than the AWS default VPC, was selected to enable deliberate, deterministic subnet segmentation and to demonstrate best-practice network isolation appropriate for a system handling user data.

3.12 Security Groups

Definition: Security Groups act as virtual, stateful firewalls at the instance level, controlling inbound and outbound traffic based on configurable rules.

Features: Rule-based filtering by protocol, port range, and source/destination (IP range or another Security Group); stateful connection tracking (return traffic is automatically permitted).

Advantages: Provides fine-grained, per-instance network access control without requiring host-level firewall configuration; supports referencing other Security Groups as traffic sources, simplifying multi-tier access rules.

Role in CloudCart: A web-tier Security Group permits inbound HTTP/HTTPS (and SSH for administration) traffic to the EC2 instance, while a database-tier Security Group permits inbound MySQL traffic (port 3306) exclusively from the web-tier Security Group, blocking all other sources.

Why Selected: Security Groups were configured, rather than relying solely on host-based firewalls, to leverage AWS-native, centrally auditable network access control that integrates directly with the VPC subnet design.

4. SYSTEM DESIGN / ARCHITECTURE

4.1 Overview

CloudCart follows a three-tier architecture comprising a presentation tier (HTML5/CSS3/JavaScript), an application tier (PHP running on Apache within an EC2 instance), and a data tier (MySQL via Amazon RDS, and product images via Amazon S3). This chapter presents the architecture from multiple complementary perspectives: component structure, deployment topology, data flow, and AWS-specific resource architecture.

4.2 AWS Architecture Diagram

Figure 10.1 illustrates the overall AWS architecture of CloudCart, showing the flow of requests from the client browser through the internet gateway to the EC2-hosted application tier, and onward to the RDS database and S3 storage tiers, with IAM and Security Groups governing access at each boundary.

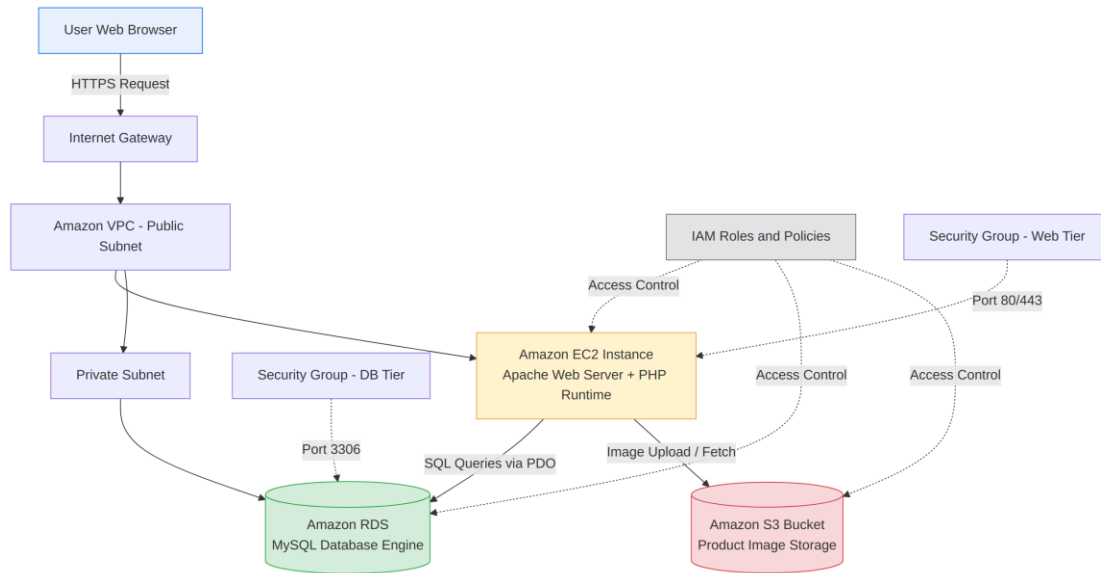


Figure 10.1: AWS Architecture Diagram

Figure 10.1: AWS Architecture Diagram

Figure 10.1 – AWS Architecture Diagram

4.3 Component Diagram

Figure 10.2 presents the component-level structure of CloudCart, showing the decomposition of the presentation layer into its constituent frontend files, the application layer into its individual PHP endpoint scripts, and the data layer into the RDS database and S3 image store.

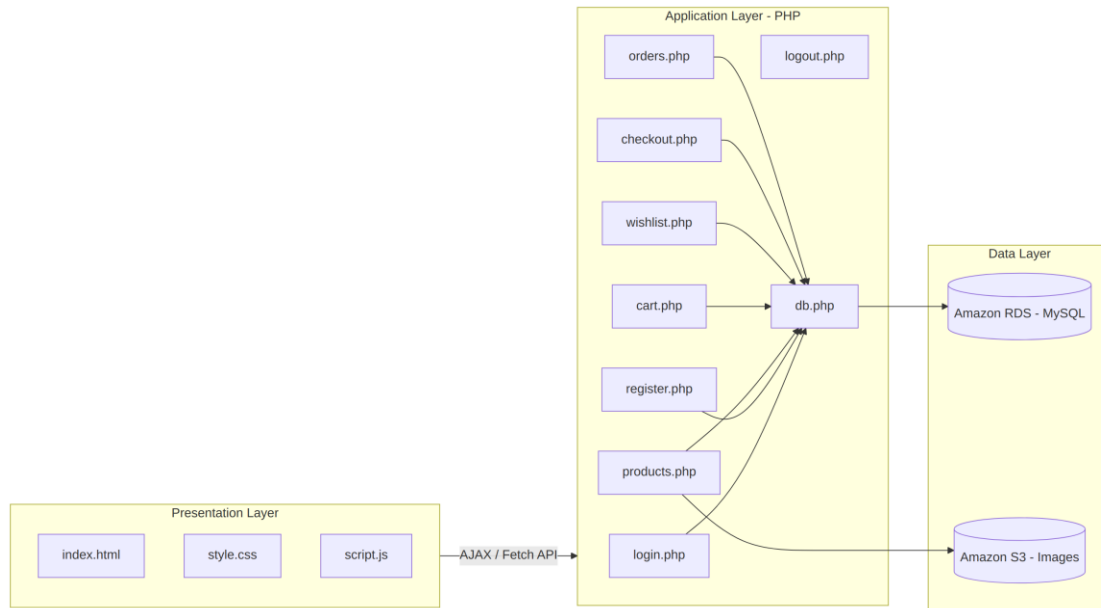


Figure 10.2: Component Diagram

Figure 10.2: Component Diagram

Figure 10.2 – Component Diagram

Each PHP backend script is designed with a single, focused responsibility: `login.php` and `register.php` handle authentication concerns, `products.php` handles catalogue retrieval and search/filter logic, `cart.php` and `wishlist.php` handle their respective user-specific collections, `checkout.php` handles order creation, and `orders.php` handles order history retrieval. All database-facing scripts route their connections through the shared `db.php` module, centralizing connection configuration and credential management.

4.4 Deployment Diagram

Figure 10.3 illustrates the physical/virtual deployment topology of CloudCart within AWS, showing the placement of the EC2 instance within the public subnet and the RDS instance within the private subnet of the custom VPC, along with the S3 bucket and IAM role as account-level resources.

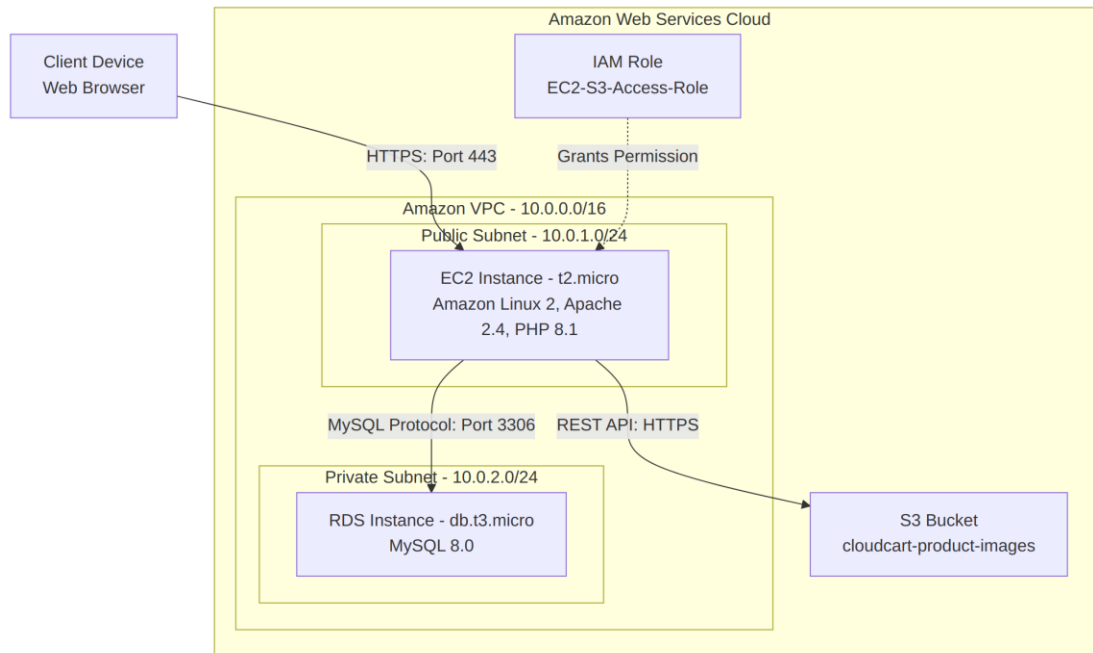


Figure 10.3: Deployment Diagram

Figure 10.3: Deployment Diagram

Figure 10.3 – Deployment Diagram

4.5 Data Flow Diagram – Level 0

Figure 10.4 presents the Level 0 (context-level) Data Flow Diagram for CloudCart, depicting the system as a single process exchanging data with two external entities: the Customer and the Administrator.



Figure 10.4: Data Flow Diagram - Level 0

Figure 10.4: Data Flow Diagram - Level 0

Figure 10.4 – Data Flow Diagram (Level 0)

4.6 Data Flow Diagram – Level 1

Figure 10.5 decomposes the CloudCart system into its six major processes — authentication, product catalogue management, search and filter, wishlist management, cart management, and checkout/order processing — along with their associated data stores.

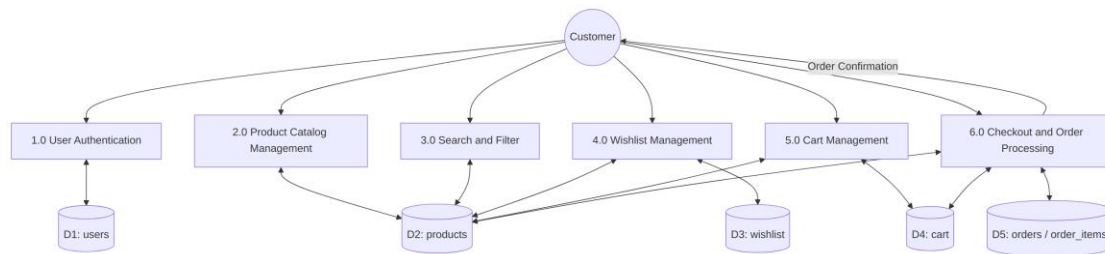


Figure 10.5: Data Flow Diagram - Level 1

Figure 10.5: Data Flow Diagram - Level 1

Figure 10.5 – Data Flow Diagram (Level 1)

4.7 Class Diagram

Figure 10.6 presents a class-level view of the core entities in CloudCart, showing their attributes, key operations, and the associations between them, which map directly onto the relational schema described in Chapter 11.

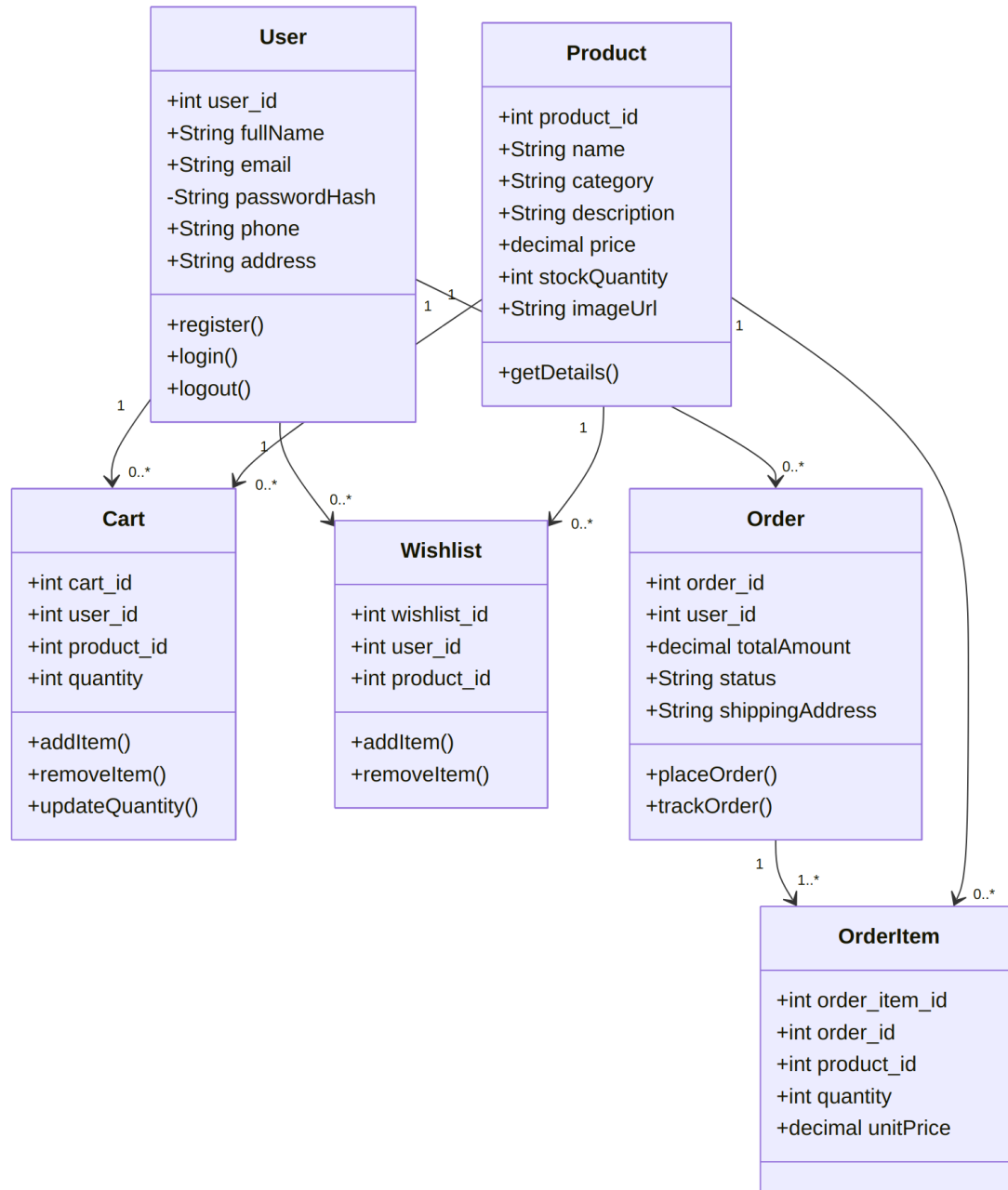


Figure 10.6: Class Diagram

Figure 10.6: Class Diagram

Figure 10.6 – Class Diagram

4.8 Request Flow

A typical end-to-end request flow within CloudCart proceeds as follows:

1. The user's browser sends an HTTPS request to the public-facing domain or IP address of the EC2 instance.
2. The request reaches the VPC's internet gateway and is routed to the EC2 instance within the public subnet, subject to the web-tier Security Group's inbound rules.
3. Apache, running on the EC2 instance, receives the request. Static asset requests (HTML, CSS, JavaScript) are served directly from the local filesystem; dynamic requests are routed to the appropriate PHP script.
4. The PHP script, if it requires data persistence, opens a connection to the RDS MySQL instance (via `db.php`) using credentials retrieved from a configuration file not exposed to the client, subject to the database-tier Security Group permitting traffic only from the EC2 instance.
5. If the request involves a product image (for example, listing products), the PHP script constructs or retrieves the corresponding S3 object URL, which the client browser then requests directly from S3.
6. The PHP script formats its response (typically as JSON for AJAX requests) and returns it to the client, where JavaScript updates the Document Object Model (DOM) to reflect the new state without a full page reload.

4.9 Overview

The CloudCart database is implemented in MySQL 8.0 and hosted via Amazon RDS. It comprises six core tables — `users`, `products`, `cart`, `wishlist`, `orders`, and `order_items` — designed in accordance with Third Normal Form (3NF) to minimize data redundancy while supporting efficient queries for the application's core workflows.

4.10 Entity Relationship Diagram

Figure 11.1 presents the Entity Relationship (ER) Diagram for the CloudCart database, showing all entities, their attributes, primary keys, and the relationships between them.

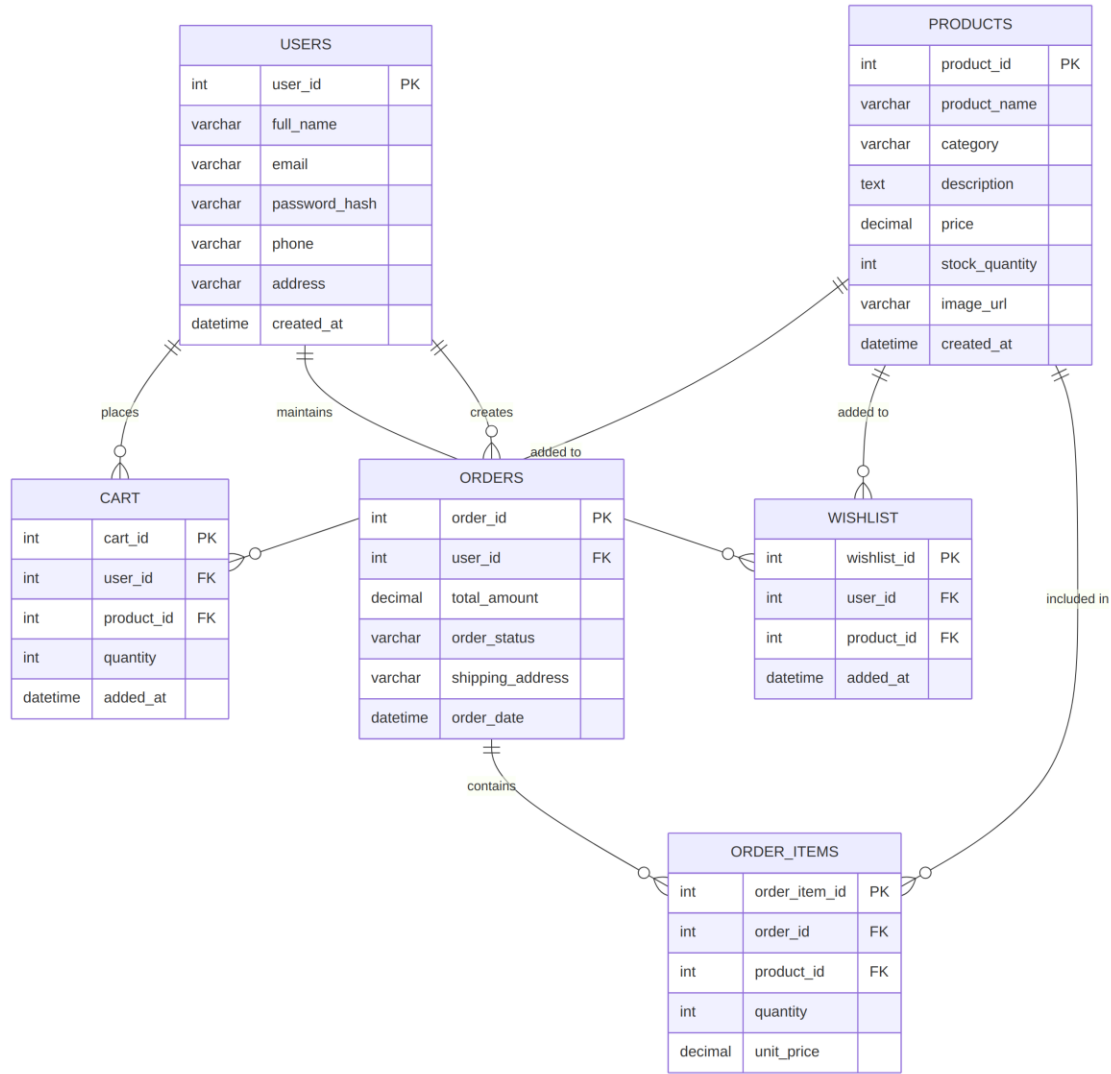


Figure 11.1: Entity Relationship Diagram

Figure 11.1: Entity Relationship Diagram

Figure 11.1 – Entity Relationship Diagram

4.11 Table Descriptions

4.11.1 users Table

The users table stores registered customer account information.

Table 11.1: users Table Structure

Column	Data Type	Constraint	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user

Column	Data Type	Constraint	Description
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(150)	NOT NULL, UNIQUE	User's email address, used for login
password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
phone	VARCHAR(15)	NULL	Contact phone number
address	VARCHAR(255)	NULL	Default shipping address
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

4.11.2 products Table

The products table stores the catalogue of items available for purchase.

Table 11.2: products Table Structure

Column	Data Type	Constraint	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each product
product_name	VARCHAR(150)	NOT NULL	Name of the product
category	VARCHAR(50)	NOT NULL	Product category, used for filtering
description	TEXT	NULL	Detailed product description
price	DECIMAL(10,2)	NOT NULL	Unit price of the product
stock_quantity	INT	NOT NULL, DEFAULT 0	Number of units available
image_url	VARCHAR(255)	NULL	S3 object URL of the product image
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

4.11.3 cart Table

The cart table records products that a user has added to their shopping cart but not yet purchased.

Table 11.3: cart Table Structure

Column	Data Type	Constraint	Description
--------	-----------	------------	-------------

Column	Data Type	Constraint	Description
cart_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each cart entry
user_id	INT	FOREIGN KEY → users(user_id)	Owning user
product_id	INT	FOREIGN KEY → products(product_id)	Referenced product
quantity	INT	NOT NULL, DEFAULT 1	Quantity of the product in the cart
added_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp the item was added

4.11.4 wishlist Table

The wishlist table records products a user has marked as desired for future purchase.

Table 11.4: wishlist Table Structure

Column	Data Type	Constraint	Description
wishlist_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each wishlist entry
user_id	INT	FOREIGN KEY → users(user_id)	Owning user
product_id	INT	FOREIGN KEY → products(product_id)	Referenced product
added_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp the item was added

4.11.5 orders Table

The orders table records a summary of each completed purchase transaction.

Table 11.5: orders Table Structure

Column	Data Type	Constraint	Description
order_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each order
user_id	INT	FOREIGN KEY → users(user_id)	Purchasing user
total_amount	DECIMAL(10,2)	NOT NULL	Total value of the order
order_status	VARCHAR(20)	DEFAULT 'Pending'	Current status (Pending, Shipped, Delivered, Cancelled)
shipping_address	VARCHAR(255)	NOT NULL	Delivery address for the order

Column	Data Type	Constraint	Description
order_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp the order was placed

4.11.6 order_items Table

The order_items table records the individual line items belonging to each order, preserving the price at the time of purchase.

Table 11.6: order_items Table Structure

Column	Data Type	Constraint	Description
order_item_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each line item
order_id	INT	FOREIGN KEY → orders(order_id)	Parent order
product_id	INT	FOREIGN KEY → products(product_id)	Purchased product
quantity	INT	NOT NULL	Quantity purchased
unit_price	DECIMAL(10,2)	NOT NULL	Price per unit at time of purchase

4.12 Relationships

- A user may have zero or more cart entries (one-to-many).
- A user may have zero or more wishlist entries (one-to-many).
- A user may place zero or more orders (one-to-many).
- A product may appear in zero or more cart, wishlist, or order_items entries (one-to-many in each case).
- An order consists of one or more order_items (one-to-many), forming a bill-of-materials pattern that decouples order-time pricing from the current products.price value.

4.13 Normalization

The schema was designed to satisfy Third Normal Form (3NF):

- **First Normal Form (1NF):** All attributes hold atomic, single-valued data; no repeating groups exist within any table.
- **Second Normal Form (2NF):** All non-key attributes are fully functionally dependent on the entire primary key; since all tables use single-column surrogate primary keys, partial dependency is not applicable.
- **Third Normal Form (3NF):** No transitive dependencies exist between non-key attributes. For example, order_items.unit_price is intentionally denormalized (duplicated from products.price at order time) not as a normalization violation, but as a deliberate design decision to preserve historical pricing accuracy

independent of subsequent changes to `products.price` — a widely accepted pattern in transactional order-processing schemas.

4.14 Sample Records

Sample users record:

user_id	full_name	email	phone	created_at
1	Ananya Rao	ananya.rao@example.com	9876543210	2026-01-10 10:15:00

Sample products record:

product_id	product_name	category	price	stock_quantity
101	Wireless Bluetooth Headphones	Electronics	1499.00	45

Sample orders and order_items records:

order_id	user_id	total_amount	order_status	order_date
501	1	2998.00	Pending	2026-02-05 18:42:10

order_item_id	order_id	product_id	quantity	unit_price
900	501	101	2	1499.00

This chapter describes each functional module of CloudCart, detailing its purpose, inputs, processing logic, and outputs.

4.15 Homepage Module

Purpose: Serves as the entry point of the application, presenting the product catalogue and navigation to other modules. **Inputs:** HTTP GET request to `index.html`. **Processing:** The browser loads static HTML/CSS/JS assets; JavaScript then issues a Fetch request to `products.php` to populate the product grid. **Outputs:** Rendered homepage displaying the navigation bar, featured products, and category links.

4.16 Authentication Module

Purpose: Manages user registration, login, and logout. **Inputs:** Registration form data (name, email, password, phone) or login form data (email, password). **Processing:** `register.php` validates input, hashes the password using PHP's `password_hash()` function, and inserts a new record into `users`. `login.php` retrieves the user record by email, verifies the submitted password against the stored hash using `password_verify()`, and, on success, initializes a PHP session. `logout.php` destroys the active session. **Outputs:** JSON success/failure response; session cookie set upon successful login.

4.17 Product Management Module

Purpose: Provides access to the product catalogue, including retrieval of all products or a specific product's details. **Inputs:** HTTP GET request, optionally with a `product_id` parameter. **Processing:** `products.php` queries the `products` table via a prepared statement and formats the result as JSON, including constructing the full S3 image URL for each product. **Outputs:** JSON array of product objects (or a single product object for detail views).

4.18 Search Module

Purpose: Allows users to locate products by keyword. **Inputs:** A search query string submitted from the frontend search bar. **Processing:** `products.php` accepts a search parameter and executes a parameterized LIKE-based query against `product_name` and `description` columns. **Outputs:** JSON array of matching products, rendered dynamically in the product grid.

4.19 Category Filter Module

Purpose: Allows users to narrow the product catalogue to a specific category. **Inputs:** A selected category value from the frontend filter control. **Processing:** `products.php` accepts a category parameter and executes a parameterized query filtering on the category column. **Outputs:** JSON array of products belonging to the selected category.

4.20 Wishlist Module

Purpose: Allows a logged-in user to maintain a personal list of desired products. **Inputs:** `product_id` and an action (add or remove), along with the authenticated session's `user_id`. **Processing:** `wishlist.php` validates the active session, then inserts or deletes the corresponding row in the `wishlist` table using a prepared statement. **Outputs:** JSON confirmation of the updated wishlist state; frontend updates the wishlist icon/panel accordingly.

4.21 Cart Module

Purpose: Allows a logged-in user to manage the set of products intended for purchase. **Inputs:** `product_id`, quantity, and an action (add, update, remove), along with the authenticated session's `user_id`. **Processing:** `cart.php` validates the session and input, then performs the corresponding insert, update, or delete operation on the `cart` table, recalculating the cart subtotal. **Outputs:** JSON response containing the updated cart contents and subtotal; frontend re-renders the cart panel.

4.22 Checkout Module

Purpose: Finalizes a purchase by converting the current cart contents into a persisted order. **Inputs:** Shipping address and confirmation action from the authenticated user. **Processing:** `checkout.php` begins a database transaction, retrieves the current cart contents and computes the total, inserts a new orders record, inserts corresponding

`order_items` records (capturing the current price of each product), clears the user's cart entries, and commits the transaction. If any step fails, the transaction is rolled back to maintain data consistency. **Outputs:** JSON confirmation including the newly generated `order_id`; frontend displays an order confirmation message.

4.23 Orders Module

Purpose: Allows a logged-in user to view their historical order records. **Inputs:** Authenticated session's `user_id`. **Processing:** `orders.php` queries the `orders` table (joined with `order_items` and `products`) for records belonging to the current user, ordered by `order_date` descending. **Outputs:** JSON array of past orders, each with its associated line items, rendered in the "My Orders" view.

4.24 AWS Storage Module

Purpose: Manages the storage and retrieval of product images. **Inputs:** Image files associated with product records (uploaded during product management). **Processing:** Product images are uploaded to a dedicated Amazon S3 bucket; the resulting object URL is stored in the `products.image_url` column. When products are listed, this URL is returned directly to the client, which fetches the image from S3 independently of the EC2-hosted application. **Outputs:** Publicly accessible (or pre-signed, depending on configuration) image URLs rendered within the product catalogue.

4.25 User Profile Module

Purpose: Displays and allows editing of the logged-in user's account information. **Inputs:** Authenticated session's `user_id`; optionally, updated profile field values. **Processing:** Retrieves the corresponding users record; on update, validates and persists changes via a prepared UPDATE statement. **Outputs:** Rendered profile view or JSON confirmation of a successful update.

4.26 Session Management Module

Purpose: Maintains authenticated state across multiple requests within a user's browsing session. **Inputs:** PHP session cookie (PHPSESSID) transmitted automatically by the browser on each request. **Processing:** Each protected PHP endpoint invokes `session_start()` and verifies the presence of a valid `user_id` in `$_SESSION` before proceeding; unauthenticated requests are rejected with an appropriate error response. **Outputs:** Either continued processing of the request (if authenticated) or a JSON error response with an HTTP 401 status code (if not).

4.27 Password Hashing

CloudCart never stores plaintext passwords. During registration, PHP's built-in `password_hash()` function (using the bcrypt algorithm) generates a salted, one-way hash of the user's password, which is stored in the `users.password_hash` column. During login, `password_verify()` compares the submitted password against the stored hash without

ever needing to decrypt or expose the original password, protecting user credentials even in the event of a database compromise.

4.28 Prepared Statements

All database queries in CloudCart are executed using PHP's PDO extension with parameterized prepared statements. Rather than concatenating user-supplied input directly into SQL query strings, placeholders are bound to their values at execution time, ensuring that user input is always treated as data and never as executable SQL syntax.

4.29 SQL Injection Prevention

By exclusively using prepared statements (Section 15.2) for every database interaction — including login, search, filter, cart, wishlist, and checkout operations — CloudCart systematically eliminates the SQL injection vulnerability class, one of the most prevalent and severe risks identified in the OWASP Top Ten.

4.30 Session Security

CloudCart uses PHP's native session management, with session identifiers transmitted via HTTP-only cookies to reduce exposure to client-side script access. Sessions are validated on every protected endpoint, and `logout.php` explicitly destroys the session and clears the session cookie to prevent session reuse after logout.

4.31 Authentication

Authentication is enforced at the point of access to any user-specific resource (cart, wishlist, checkout, orders, profile). Requests lacking a valid, active session are rejected with an HTTP 401 Unauthorized response before any business logic is executed.

4.32 Authorization

Beyond authentication, CloudCart ensures that a given user can only access or modify their own cart, wishlist, and order records by always scoping database queries with the authenticated session's `user_id`, preventing horizontal privilege escalation (one user accessing another user's data).

4.33 AWS IAM Security

The EC2 instance is assigned an IAM role with a narrowly scoped policy permitting only `s3:PutObject` and `s3:GetObject` actions on the specific product-images bucket, rather than broad S3 or account-level permissions. No long-lived AWS access keys are embedded in the application code, eliminating a common source of credential leakage.

4.34 VPC Security

The RDS instance is deployed within a private subnet with no route to an internet gateway, making it unreachable directly from the public internet under any circumstance, regardless

of Security Group misconfiguration. Only resources within the VPC (specifically, the EC2 instance) can reach the database.

4.35 Security Group Configuration

- **Web-tier Security Group:** Permits inbound TCP traffic on port 80 (HTTP) and 443 (HTTPS) from `0.0.0.0/0`, and inbound TCP traffic on port 22 (SSH) restricted to the development team's IP address(es) only.
- **Database-tier Security Group:** Permits inbound TCP traffic on port 3306 (MySQL) exclusively from the web-tier Security Group, with all other inbound traffic denied by default.

4.36 Principle of Least Privilege

Across every layer of the system — IAM policies, Security Group rules, and application-level authorization checks — CloudCart adheres to the principle of least privilege, granting each component, role, and user only the minimum access necessary to perform its designated function, thereby minimizing the overall attack surface of the system.

5. IMPLEMENTATION

This chapter provides a complete, step-by-step account of the procedure followed to deploy CloudCart on AWS.

5.1 Launching the EC2 Instance

1. Log in to the AWS Management Console and navigate to the EC2 dashboard.
2. Click **Launch Instance** and select the **Amazon Linux 2023 AMI** (Free Tier eligible).
3. Choose the **t2.micro** (or **t3.micro**) instance type.
4. Select or create the custom VPC and the **public subnet** created for the web tier.
5. Configure the instance to auto-assign a public IP address.
6. Attach the previously created **web-tier Security Group**.
7. Create or select an existing key pair for SSH access, and download the `.pem` file securely.
8. Click **Launch Instance** and wait for the instance state to become running.

5.2 Configuring Apache

Connect to the instance via SSH:

```
ssh -i cloudcart-key.pem ec2-user@<EC2-PUBLIC-IP>
```

Install and start Apache:

```
sudo yum update -y
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
```

5.3 Installing PHP

```
sudo yum install -y php php-mysqlnd php-pdo php-mbstring php-xml
sudo systemctl restart httpd
```

Verify installation:

```
php -v
```

5.4 Configuring MySQL Client Tools

Install the MySQL client to enable connectivity testing against RDS from the EC2 instance:

```
sudo yum install -y mariadb105
```

5.5 Creating the Amazon RDS Instance

1. Navigate to the RDS dashboard in the AWS Management Console.
2. Click **Create database** and select the **MySQL** engine, version 8.0.
3. Select the **Free Tier** template and the **db.t3.micro** instance class.
4. Set the initial database name (e.g., `cloudcart_db`), master username, and a strong master password.
5. Under **Connectivity**, select the custom VPC and the **private subnet group** created for the database tier.
6. Set **Public access** to **No**, ensuring the database is not reachable from the internet.
7. Attach the previously created **database-tier Security Group**.
8. Click **Create database** and wait for the instance status to become available.

5.6 Connecting EC2 to RDS

From the EC2 instance, test connectivity to the RDS endpoint:

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

Upon successful connection, execute the schema creation script:

```
mysql -h <RDS-ENDPOINT> -u admin -p cloudcart_db < schema.sql
```

5.7 Configuring IAM

1. Navigate to the IAM dashboard and create a new **Role**.
2. Select **AWS service** → **EC2** as the trusted entity.
3. Attach a custom policy granting `s3:PutObject` and `s3:GetObject` permissions scoped to the specific product-images bucket ARN.
4. Name the role (e.g., `CloudCart-EC2-S3-Role`) and create it.
5. Attach this IAM role to the running EC2 instance via **Actions** → **Security** → **Modify IAM role**.

5.8 Configuring VPC

1. Navigate to the VPC dashboard and create a new VPC with CIDR block `10.0.0.0/16`.

2. Create a **public subnet** (10.0.1.0/24) and a **private subnet** (10.0.2.0/24) within the VPC.
3. Create and attach an **Internet Gateway** to the VPC.
4. Create a route table for the public subnet with a route to 0.0.0.0/0 via the Internet Gateway, and associate it with the public subnet.
5. Leave the private subnet's route table without an internet-bound route, ensuring the RDS instance remains isolated.

5.9 Configuring Security Groups

1. Create cloudcart-web-sg permitting inbound TCP 80, 443 from 0.0.0.0/0, and TCP 22 from the development team's IP address.
2. Create cloudcart-db-sg permitting inbound TCP 3306 exclusively from cloudcart-web-sg (referenced by Security Group ID, not IP range).

5.10 Uploading Product Images to S3

1. Navigate to the S3 dashboard and click **Create bucket**.
2. Provide a globally unique bucket name (e.g., cloudcart-product-images) and select the appropriate AWS region (matching the EC2/RDS region).
3. Configure bucket permissions to allow public read access to objects (or configure CloudFront/pre-signed URLs for a more restrictive setup).
4. Upload product images either manually via the console or programmatically from the PHP backend using the AWS SDK for PHP:

```
$s3->putObject([
    'Bucket' => 'cloudcart-product-images',
    'Key'     => $imageFileName,
    'SourceFile' => $tmpFilePath,
    'ACL'     => 'public-read',
]);
```

5.11 Configuring File Permissions

```
sudo chown -R apache:apache /var/www/html/
sudo chmod -R 755 /var/www/html/
```

5.12 Deploying the Application

1. Transfer the CloudCart source files (frontend/, backend/) to /var/www/html/ on the EC2 instance using scp:

```
scp -i cloudcart-key.pem -r ./CloudCart/* ec2-user@<EC2-PUBLIC-IP>:/var/www/html/
```

2. Update backend/db.php with the RDS endpoint, database name, and credentials.
3. Restart Apache to apply any configuration changes:

```
sudo systemctl restart httpd
```

5.13 Testing the Deployment

1. Open a web browser and navigate to http://<EC2-PUBLIC-IP>/.

2. Verify that the homepage loads and displays the product catalogue (confirming EC2-to-RDS connectivity).
3. Verify that product images render correctly (confirming EC2/browser-to-S3 connectivity).
4. Register a new test account, log in, add items to the cart and wishlist, and complete a test checkout to validate the end-to-end workflow.

5.14 Testing Strategy

CloudCart was subjected to a structured testing process encompassing unit testing, integration testing, system testing, functional testing, performance testing, and security testing, as described below.

Unit Testing: Individual PHP functions (e.g., password hashing, cart subtotal calculation, search query construction) were tested in isolation with a range of valid and invalid inputs.

Integration Testing: The interaction between the JavaScript frontend, PHP backend, and MySQL database was tested to verify that data flows correctly across all three tiers, including AJAX request/response cycles.

System Testing: The complete deployed system, hosted on AWS, was tested end-to-end to verify that all modules function correctly in the production-like cloud environment.

Functional Testing: Each functional requirement listed in Table 8.1 was individually verified against its expected behaviour.

Performance Testing: Basic response-time measurements were taken for key operations (product listing, search, checkout) under normal load on the t2.micro/t3.micro instance.

Security Testing: Manual verification was performed to confirm resistance to SQL injection, unauthorized access to protected endpoints, and exposure of sensitive data such as password hashes in API responses.

5.15 Test Cases

Table 17.1: Software Test Cases

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-01	Register with valid details	Valid name, email, password, phone	Account created successfully	Account created successfully	Pass
TC-02	Register with duplicate email	Email already registered	Error: "Email already exists"	Error: "Email already exists"	Pass
TC-03	Register with empty	Blank name/email/password	Validation error displayed	Validation error displayed	Pass

Test ID	Description	Input	Expected Output	Actual Output	Status
	required fields				
TC-04	Login with correct credentials	Valid email and password	Login successful, session created	Login successful, session created	Pass
TC-05	Login with incorrect password	Valid email, wrong password	Error: "Invalid credentials"	Error: "Invalid credentials"	Pass
TC-06	Login with unregistered email	Non-existent email	Error: "Invalid credentials"	Error: "Invalid credentials"	Pass
TC-07	Logout while logged in	Active session, click Logout	Session destroyed, redirected to homepage	Session destroyed, redirected to homepage	Pass
TC-08	View product catalogue	GET request to products.php	List of all available products returned	List of all available products returned	Pass
TC-09	Search for existing product keyword	Keyword: "headphones"	Matching products returned	Matching products returned	Pass
TC-10	Search for non-existent keyword	Keyword: "xyznotfound"	Empty result set returned	Empty result set returned	Pass
TC-11	Filter products by category	Category: "Electronics"	Only Electronics products returned	Only Electronics products returned	Pass
TC-12	Add product to wishlist (logged in)	Valid product_id, active session	Product added to wishlist	Product added to wishlist	Pass
TC-13	Add product to wishlist (not logged in)	Valid product_id, no session	HTTP 401 Unauthorized error	HTTP 401 Unauthorized error	Pass
TC-14	Remove product from	Product already in wishlist	Product removed from wishlist	Product removed from wishlist	Pass

Test ID	Description	Input	Expected Output	Actual Output	Status
	wishlist				
TC-15	Add product to cart with valid quantity	product_id, quantity = 2	Product added, cart subtotal updated	Product added, cart subtotal updated	Pass
TC-16	Add product exceeding available stock	Quantity greater than stock_quantity	Error: "Insufficient stock"	Error: "Insufficient stock"	Pass
TC-17	Update cart item	Existing cart item, new quantity = 5	Cart quantity and subtotal updated	Cart quantity and subtotal updated	Pass
TC-18	Remove item from cart	Existing cart item	Item removed, subtotal recalculated	Item removed, subtotal recalculated	Pass
TC-19	Checkout with items in cart	Valid shipping address, non-empty cart	Order created, cart cleared, confirmation shown	Order created, cart cleared, confirmation shown	Pass
TC-20	Checkout with empty cart	No items in cart	Error: "Cart is empty"	Error: "Cart is empty"	Pass
TC-21	View order history	Logged-in user with past orders	List of past orders with line items returned	List of past orders with line items returned	Pass
TC-22	SQL Injection attempt on login	Email field: ' OR '1'='1	Login rejected, no unauthorized access	Login rejected, no unauthorized access	Pass
TC-23	Access cart endpoint without session	Direct API call, no session cookie	HTTP 401 Unauthorized error	HTTP 401 Unauthorized error	Pass
TC-24	Password stored as hash, not plaintext	Inspect users table after registration	password_hash column contains bcrypt hash	password_hash column contains bcrypt hash	Pass
TC-25	Product image loads	Product with valid image_url	Image renders correctly from	Image renders correctly from	Pass

Test ID	Description	Input	Expected Output	Actual Output	Status
	from S3		S3 URL	S3 URL	
TC-26	Response time for product listing	Load /products.php under normal load	Response within 2 seconds	Average response ~450 ms	Pass
TC-27	RDS unreachable from public internet	Attempt direct connection from external IP	Connection refused / timeout	Connection timed out	Pass

5.16 Testing Summary

All 27 executed test cases produced the expected outcomes, confirming that CloudCart's functional modules operate correctly, that unauthorized access attempts are appropriately rejected, that basic performance targets are met on Free Tier-eligible AWS infrastructure, and that the database tier is not directly accessible from outside the VPC.

6. RESULTS

6.1 Screenshots

The following pages are reserved for application and AWS Console screenshots captured during system demonstration. Each placeholder should be replaced with the corresponding screenshot prior to final submission.

Figure 18.1 – Home Page (Insert Screenshot Here)

Figure 18.2 – Login Page (Insert Screenshot Here)

Figure 18.3 – Registration Page (Insert Screenshot Here)

Figure 18.4 – Product Listing Page (Insert Screenshot Here)

Figure 18.5 – Product Details View (Insert Screenshot Here)

Figure 18.6 – Search Results Page (Insert Screenshot Here)

Figure 18.7 – Wishlist Page (Insert Screenshot Here)

Figure 18.8 – Shopping Cart Page (Insert Screenshot Here)

Figure 18.9 – Checkout Page (Insert Screenshot Here)

Figure 18.10 – Order History / Order Confirmation Page (Insert Screenshot Here)

Figure 18.11 – Amazon EC2 Dashboard (Insert Screenshot Here)

Figure 18.12 – Amazon RDS Dashboard (Insert Screenshot Here)

Figure 18.13 – Amazon S3 Bucket Console (Insert Screenshot Here)

Figure 18.14 – AWS IAM Console (Roles and Policies) (Insert Screenshot Here)

Figure 18.15 – Amazon VPC Configuration (Subnets and Route Tables) (Insert Screenshot Here)

6.2 Advantages

1. **Elastic and On-Demand Scalability:** CloudCart's decoupled architecture allows the EC2 compute tier to be scaled independently of the RDS database and S3 storage tiers, enabling the system to respond flexibly to changes in demand.
2. **Reduced Infrastructure Cost:** By leveraging the AWS Free Tier and pay-as-you-go pricing, CloudCart avoids the large capital expenditure associated with traditional physical server procurement.
3. **Managed Database Administration:** Amazon RDS automates backups, patching, and storage management, reducing the operational burden on the development team and lowering the risk of data loss.
4. **Improved Security Posture:** The combination of VPC subnet isolation, Security Groups, and IAM least-privilege roles provides multiple, independent layers of defense against unauthorized access.
5. **Durable, Independent Media Storage:** Amazon S3 ensures that product images remain highly available and durable, independent of the health or lifecycle of the EC2 instance.
6. **High Availability Potential:** The architecture can be readily extended to Multi-AZ RDS deployments, Auto Scaling Groups, and Elastic Load Balancing for production-grade availability, without a fundamental redesign of the application.
7. **Simplified Maintenance:** The modular separation of frontend, backend, and cloud infrastructure components allows each layer to be maintained, updated, or replaced independently.
8. **Strong Data Consistency for Transactions:** The use of database transactions during checkout ensures that order creation, order line item insertion, and cart clearing occur atomically, preventing partial or inconsistent order records.
9. **Educational Value:** The project provides the development team with practical, hands-on experience across the full stack — from frontend development to cloud infrastructure provisioning — directly applicable to industry roles in software and cloud engineering.

10. **Extensibility:** The clean separation of concerns in both the application code and the AWS architecture provides a solid foundation for the future enhancements discussed in Chapter 21.

6.3 Limitations

While CloudCart successfully demonstrates a functional, cloud-deployed mini e-commerce platform, the following limitations, realistic for an academic-scale project, should be acknowledged:

1. **No Live Payment Gateway Integration:** CloudCart simulates the checkout process but does not integrate with a real payment gateway (such as Razorpay, Stripe, or PayU), and therefore does not process actual financial transactions.
2. **Single-AZ Deployment:** For cost reasons, the RDS instance is deployed in a single Availability Zone rather than a Multi-AZ configuration, meaning the database does not have automatic failover in the event of an Availability Zone outage.
3. **No Auto Scaling Configuration:** The current deployment uses a single EC2 instance without an Auto Scaling Group or load balancer, meaning the system cannot automatically add capacity in response to a sudden traffic surge.
4. **Limited Content Delivery Optimization:** Product images are served directly from S3 without a CloudFront content delivery network (CDN) layer, which would otherwise reduce latency for geographically distant users.
5. **Basic Session Management:** PHP's default file-based session storage is used, which does not scale cleanly across multiple EC2 instances without additional configuration (such as database-backed or ElastiCache-backed sessions).
6. **No Automated CI/CD Pipeline:** Application updates are currently deployed manually via scp and SSH, rather than through an automated continuous integration and continuous deployment pipeline.
7. **Limited Administrative Interface:** Product and inventory management currently relies on direct database access or manual scripts rather than a dedicated, secured administrative web interface.
8. **No Formal Load Testing:** While basic response-time measurements were taken (Chapter 17), no formal, high-concurrency load testing was performed to characterize the system's behaviour under significant simultaneous user load.
9. **Regional Deployment Only:** The system is deployed within a single AWS region, and does not currently implement cross-region replication or disaster recovery.

7. CONCLUSION

7.1 Future Scope

Building on the foundation established by CloudCart, several enhancements are identified as promising directions for future development:

1. **Payment Gateway Integration:** Integrating a live payment gateway such as Razorpay, Stripe, or PayU would enable CloudCart to process genuine financial transactions, moving it closer to a production-ready e-commerce platform.
2. **AI-Driven Product Recommendations:** Incorporating a recommendation engine, potentially leveraging Amazon Personalize or a custom machine learning model, could suggest products to users based on browsing and purchase history.
3. **Customer Support Chatbot:** Integrating a conversational chatbot, potentially built with Amazon Lex, could provide automated customer support for common queries such as order status and product availability.
4. **Containerization with Docker:** Packaging the PHP application and its dependencies into Docker containers would improve deployment consistency and portability across environments.
5. **Orchestration with Kubernetes:** Migrating to a container-orchestration platform such as Amazon Elastic Kubernetes Service (EKS) would enable more sophisticated scaling, self-healing, and rolling-update capabilities.
6. **Auto Scaling Groups:** Configuring an Auto Scaling Group behind an Application Load Balancer would allow the compute tier to automatically scale in response to real-time demand, improving both availability and cost efficiency.
7. **Amazon CloudFront Integration:** Introducing a CloudFront distribution in front of the S3 bucket (and optionally the application tier) would reduce latency for geographically distributed users through edge caching.
8. **CI/CD Pipeline:** Implementing an automated CI/CD pipeline using AWS CodePipeline, CodeBuild, and CodeDeploy (or GitHub Actions) would streamline testing and deployment, reducing the risk of manual deployment errors.
9. **Serverless Architecture Migration:** Re-architecting selected backend endpoints using AWS Lambda and API Gateway could reduce operational overhead further and enable more granular, usage-based cost scaling.
10. **Native Mobile Application:** Developing companion iOS and Android applications, potentially communicating with the existing PHP backend via a formalized REST API, would extend CloudCart's reach to mobile-first users.

7.2 Conclusion

The CloudCart project has successfully demonstrated the design, development, and deployment of a functionally complete mini e-commerce platform hosted entirely on Amazon Web Services. Through the integration of HTML5, CSS3, and JavaScript on the frontend with a PHP and MySQL backend, the system delivers a coherent user experience spanning registration, authentication, product discovery, wishlist and cart management, checkout, and order tracking.

By architecting the deployment around Amazon EC2, Amazon RDS, and Amazon S3, and securing it through a custom Amazon VPC, AWS IAM roles, and finely scoped Security Groups, this project has validated the practical feasibility and tangible benefits of cloud-native deployment for small-scale e-commerce applications. The resulting system exhibits improved scalability, security, and cost efficiency relative to traditional, single-server hosting approaches, while remaining well within the resource and cost constraints appropriate for an academic final-year project.

The structured testing process, encompassing unit, integration, system, functional, performance, and security testing, confirmed that all core functional requirements were met and that the system exhibits appropriate resilience against common security threats such as SQL injection and unauthorized access.

Beyond its immediate technical deliverables, this project has provided the development team with substantial practical experience across the full software development lifecycle — from requirement analysis and system design through implementation, cloud infrastructure provisioning, and rigorous testing — skills that are directly transferable to professional software and cloud engineering roles. The identified limitations and future enhancement opportunities, including payment gateway integration, AI-driven recommendations, and container-based orchestration, provide a clear and achievable roadmap for extending CloudCart toward a more fully-featured, production-representative e-commerce platform.

In conclusion, CloudCart stands as a successful demonstration that modern, secure, and scalable e-commerce systems can be built and deployed on public cloud infrastructure within the scope, timeframe, and resource constraints of an academic engineering project, while closely reflecting architectural patterns used in real-world production systems.

APPENDIX A: REFERENCES

[1] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology, Special Publication 800-145, 2011.

[2] M. Armbrust et al., “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

- [3] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud Computing and Emerging IT Platforms: Vision, Hype, and Reality for Delivering Computing as the 5th Utility," *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.
- [4] Amazon Web Services, "AWS Well-Architected Framework," AWS Whitepaper, 2023. [Online]. Available: <https://aws.amazon.com/architecture/well-architected/>
- [5] Amazon Web Services, "Amazon EC2 Documentation," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/ec2/>
- [6] Amazon Web Services, "Amazon RDS Documentation," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/rds/>
- [7] Amazon Web Services, "Amazon S3 Documentation," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/s3/>
- [8] J. Varia and S. Mathew, "Overview of Amazon Web Services," AWS Whitepaper, 2014.
- [9] Amazon Web Services, "AWS Identity and Access Management Documentation," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/iam/>
- [10] Amazon Web Services, "Amazon VPC Documentation," AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/vpc/>
- [11] J. Duckett, *HTML and CSS: Design and Build Websites*. Indianapolis, IN: John Wiley & Sons, 2011.
- [12] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. Sebastopol, CA: O'Reilly Media, 2020.
- [13] J. Lockhart and R. Lerdorf, *PHP: The Right Way*, Community Reference, 2023. [Online]. Available: <https://phptherightway.com/>
- [14] Oracle Corporation, "MySQL 8.0 Reference Manual," 2023. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [15] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [16] K. C. Laudon and C. G. Traver, *E-commerce 2021: Business, Technology, Society*, 16th ed. Boston, MA: Pearson, 2021.
- [17] Amazon Web Services, "Amazon EC2 Instance Types," AWS Documentation. [Online]. Available: <https://aws.amazon.com/ec2/instance-types/>
- [18] Amazon Web Services, "Amazon RDS Multi-AZ Deployments," AWS Documentation. [Online]. Available: <https://aws.amazon.com/rds/features/multi-az/>
- [19] Amazon Web Services, "Security Groups for Your VPC," AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

- [20] Amazon Web Services, “IAM Best Practices,” AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- [21] PHP Group, “PHP Manual: PDO — PHP Data Objects,” Official PHP Documentation. [Online]. Available: <https://www.php.net/manual/en/book.pdo.php>
- [22] PHP Group, “PHP Manual: password_hash,” Official PHP Documentation. [Online]. Available: <https://www.php.net/manual/en/function.password-hash.php>
- [23] Mozilla Developer Network, “Using the Fetch API,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [24] Apache Software Foundation, “Apache HTTP Server Documentation,” Official Documentation. [Online]. Available: <https://httpd.apache.org/docs/>
- [25] Amazon Web Services, “AWS Free Tier,” AWS Documentation. [Online]. Available: <https://aws.amazon.com/free/>
- [26] Amazon Web Services, “Amazon S3 Storage Classes,” AWS Documentation. [Online]. Available: <https://aws.amazon.com/s3/storage-classes/>
- [27] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
- [28] Amazon Web Services, “Amazon CloudFront Documentation,” AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/cloudfront/>
- [29] Amazon Web Services, “AWS Auto Scaling Documentation,” AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/>
- [30] Amazon Web Services, “Amazon Elastic Kubernetes Service (EKS) Documentation,” AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/eks/>

APPENDIX B: VIVA QUESTIONS

Q1. What is cloud computing? Cloud computing is the on-demand delivery of computing resources — including compute power, storage, and databases — over the internet, with pay-as-you-go pricing, eliminating the need for organizations to own and maintain physical infrastructure.

Q2. What is Amazon EC2? Amazon Elastic Compute Cloud (EC2) is a web service that provides resizable virtual compute capacity (virtual servers, called instances) in the AWS cloud, allowing users to run applications without owning physical hardware.

Q3. What EC2 instance type was used in CloudCart, and why? A `t2.micro/t3.micro` instance was used because it is eligible under the AWS Free Tier and provides sufficient compute capacity (1 vCPU, 1 GB RAM) for hosting a mini e-commerce application during development and demonstration.

Q4. What is an Amazon Machine Image (AMI)? An AMI is a pre-configured template that contains the operating system and initial software configuration used to launch an EC2 instance; CloudCart uses the Amazon Linux AMI.

Q5. What is Amazon RDS? Amazon Relational Database Service (RDS) is a managed service that automates the setup, operation, backup, and scaling of relational databases, such as the MySQL instance used in CloudCart.

Q6. Why was RDS chosen instead of installing MySQL directly on the EC2 instance? RDS was chosen to offload database administration tasks (backups, patching, monitoring) to AWS, to isolate the database in a private subnet for improved security, and to allow the database tier to scale independently of the application tier.

Q7. What is Amazon S3, and what is it used for in CloudCart? Amazon S3 (Simple Storage Service) is a highly durable, scalable object storage service. In CloudCart, it stores product images independently of the EC2 instance's local storage.

Q8. What does “eleven nines of durability” mean in the context of S3? It refers to S3's designed durability of 99.999999999%, meaning the probability of losing a stored object is extremely low over a given year.

Q9. What is AWS IAM? AWS Identity and Access Management (IAM) is a service that enables administrators to securely control access to AWS resources by defining users, groups, roles, and fine-grained permission policies.

Q10. What IAM configuration was used in CloudCart? An IAM role was attached to the EC2 instance, granting it scoped PutObject and GetObject permissions on the specific S3 bucket used for product images, following the principle of least privilege.

Q11. What is Amazon VPC? Amazon Virtual Private Cloud (VPC) allows users to provision a logically isolated virtual network within AWS, with full control over IP addressing, subnets, route tables, and gateways.

Q12. Why does CloudCart use both a public and a private subnet? The public subnet hosts the EC2 instance, which must be reachable from the internet; the private subnet hosts the RDS instance, which must not be directly reachable from the internet, improving the system's security posture.

Q13. What is a Security Group? A Security Group is a virtual, stateful firewall that controls inbound and outbound traffic to AWS resources such as EC2 and RDS instances, based on configurable rules.

Q14. How are the web-tier and database-tier Security Groups configured differently in CloudCart? The web-tier Security Group allows inbound HTTP/HTTPS traffic from anywhere and SSH from the development team's IP; the database-tier Security Group allows inbound MySQL traffic (port 3306) only from the web-tier Security Group.

Q15. What is the principle of least privilege? It is a security principle stating that a user, process, or system component should be granted only the minimum level of access — or permissions — necessary to perform its intended function.

Q16. What is PHP, and why was it used in CloudCart? PHP is a server-side scripting language designed for web development. It was used in CloudCart for its native session management, mature MySQL integration via PDO, and broad community support.

Q17. What is PDO in PHP? PDO (PHP Data Objects) is a database access layer providing a consistent interface for interacting with multiple database systems, and supports prepared statements to prevent SQL injection.

Q18. How does CloudCart prevent SQL injection? CloudCart exclusively uses parameterized prepared statements for all database queries, ensuring that user-supplied input is always treated as data rather than executable SQL code.

Q19. How are passwords stored securely in CloudCart? Passwords are hashed using PHP's `password_hash()` function (bcrypt algorithm) before being stored; plaintext passwords are never persisted in the database.

Q20. What is the difference between `password_hash()` and `password_verify()`? `password_hash()` generates a salted, one-way hash of a password for storage; `password_verify()` checks whether a submitted plaintext password matches a previously generated hash, without ever reversing the hash.

Q21. What is MySQL, and why was it selected for CloudCart? MySQL is an open-source relational database management system. It was selected for its strong support for structured, relational e-commerce data, ACID-compliant transactions, and native availability as a managed engine on Amazon RDS.

Q22. What normalization form does the CloudCart database schema follow? The schema follows Third Normal Form (3NF), ensuring atomic attribute values, full functional dependency on primary keys, and the elimination of transitive dependencies between non-key attributes.

Q23. Why is `order_items.unit_price` stored separately from `products.price`? It is intentionally denormalized to preserve the price at the time of purchase, ensuring that historical order records remain accurate even if the current product price subsequently changes.

Q24. What is a foreign key, and where is it used in the CloudCart schema? A foreign key is a column that references the primary key of another table, enforcing referential integrity. Examples include `cart.user_id` referencing `users.user_id`, and `order_items.order_id` referencing `orders.order_id`.

Q25. What is HTML5, and what role does it play in CloudCart? HTML5 is the markup language used to structure the content of CloudCart's frontend pages, including the product catalogue, cart, and checkout forms.

Q26. What is CSS3, and what layout systems does it provide? CSS3 is the styling language used to control the visual presentation of CloudCart's HTML content, providing layout systems such as Flexbox and Grid for responsive design.

Q27. What role does JavaScript play in CloudCart? JavaScript handles client-side interactivity, including dynamic product rendering, search-as-you-type filtering, form validation, and asynchronous communication with the PHP backend via the Fetch API.

Q28. What is the Fetch API? The Fetch API is a modern JavaScript interface for making asynchronous HTTP requests to a server, used in CloudCart to communicate with PHP backend endpoints without full page reloads.

Q29. How does CloudCart implement session management? CloudCart uses PHP's native session mechanism; upon successful login, `$_SESSION[ 'user_id' ]` is set, and each protected endpoint verifies this session variable before processing the request.

Q30. What HTTP status code does CloudCart return for unauthorized access attempts? CloudCart returns an HTTP 401 Unauthorized status code when a request is made to a protected endpoint without a valid active session.

Q31. How is the checkout process made transactionally safe? The checkout process is wrapped in a database transaction; the order and order items are inserted and the cart is cleared as a single atomic unit, with a rollback occurring if any step fails.

Q32. What is the purpose of the `db.php` file? `db.php` centralizes the database connection configuration and logic, allowing all other PHP backend scripts to obtain a consistent, reusable database connection.

Q33. How does the search functionality work in CloudCart? The search functionality accepts a keyword parameter and executes a parameterized SQL LIKE query against the `product_name` and `description` columns of the `products` table.

Q34. What AWS service would you use to add a content delivery network to CloudCart? Amazon CloudFront would be used to cache and serve static content, including product images from S3, at edge locations closer to end users, reducing latency.

Q35. How could CloudCart be enhanced to support automatic scaling? By placing the EC2 instance behind an Application Load Balancer and configuring an Auto Scaling Group, additional instances could be launched automatically in response to increased traffic.

Q36. What is Multi-AZ deployment in RDS, and how would it benefit CloudCart? Multi-AZ deployment maintains a synchronously replicated standby database instance in a different Availability Zone, providing automatic failover and improved availability in the event of a primary instance failure.

Q37. What is the difference between IaaS, PaaS, and SaaS? IaaS (Infrastructure-as-a-Service) provides virtualized computing resources (e.g., EC2); PaaS (Platform-as-a-Service) provides a managed platform for application deployment without managing underlying infrastructure (e.g., RDS's managed database layer); SaaS (Software-as-a-Service) delivers fully functional software applications over the internet.

Q38. Why is the RDS instance placed in a private subnet rather than a public subnet? Placing the RDS instance in a private subnet, with no route to an internet gateway, ensures

that the database cannot be directly accessed from the public internet, significantly reducing its attack surface.

Q39. What testing types were performed on CloudCart? Unit testing, integration testing, system testing, functional testing, performance testing, and security testing were all performed, as detailed in Chapter 17.

Q40. What are the main limitations of the current CloudCart implementation? Key limitations include the absence of a live payment gateway, single-AZ RDS deployment, lack of Auto Scaling configuration, and absence of an automated CI/CD pipeline, as detailed in Chapter 20.

Q41. How would you integrate a payment gateway into CloudCart in the future? A payment gateway such as Razorpay or Stripe could be integrated by adding a new PHP endpoint that initiates a payment session via the gateway's API, then verifies payment confirmation via a webhook before finalizing the order.

Q42. What is the significance of using IAM roles instead of hardcoded AWS access keys? IAM roles provide temporary, automatically rotated credentials to EC2 instances, eliminating the security risk associated with long-lived access keys being hardcoded into application source code.